PROJECT REPORT

Smart Image Processing Toolbox for Real-World Applications

License Plate Recognition Using SVM

A MATLAB toolbox for image enhancement, segmentation and geometric processing, applied to licence plate recognition with a support vector machine.

Student	Name	Student ID	Responsibility
Student 1	Name of Student 1	ID	Module 1: image enhancement and filtering (spatial and frequency domains)
Student 2	Name of Student 2	ID	Module 2: segmentation, edge detection and morphological operations
Student 3	Name of Student 3	ID	Module 3: geometric transformations, interpolation, integration and testing

Item	Details
Group	Group number
Course	Course name and code
Instructor	Instructor name
Institution	University / faculty
Submission date	Date

Contents

Abstract 4
1 Introduction 5
1.1 Background and motivation 5
1.2 Project objectives 5
1.3 Scope and constraints 5
1.4 Team structure and responsibilities 6
1.5 Organisation of the report 6
2 System overview 7
2.1 Architecture 7
2.2 Software structure 7
2.3 Design conventions 8
2.4 Development and test environment 8
3 Module 1: Image Enhancement 9
3.1 Objectives 9
3.2 Theory 9
3.3 MATLAB implementation 11
3.4 Results 11
3.5 Discussion 14
4 Module 2: Segmentation and Edge Detection 15
4.1 Objectives 15
4.2 Theory 15
4.3 MATLAB implementation 16
4.4 Results 17
4.5 Discussion 21
5 Module 3: Geometric Transformations and Interpolation 22
5.1 Objectives 22
5.2 Theory 22
5.3 MATLAB implementation 23
5.4 Results 24
5.5 Discussion 27
6 License Plate Recognition with an SVM 29
6.1 Problem statement and overview 29
6.2 Plate detection 29
6.3 Character segmentation 30
6.4 Feature extraction: HOG 31
6.5 Support vector machine classifier 31
6.6 Training data 33
6.7 Training and model selection 35

6.8 End-to-end evaluation 36
6.9 Error analysis 37
7 Graphical User Interface 40
7.1 Overview 40
7.2 Implementation 42
8 Testing and Validation 44
8.1 Strategy 44
8.2 Checks 44
8.3 Defects found and fixed 44
9 Discussion 46
9.1 Summary of findings 46
9.2 Limitations 46
9.3 Applications of the toolbox in real-world scenarios 47
10 Challenges and Lessons Learned 48
10.1 Technical challenges 48
10.2 Team collaboration 48
10.3 Lessons learned 48
11 Conclusion and Future Work 49
11.1 Conclusion 49
11.2 Future work 49
References 50
Appendix A: Reproducing the Results 51
Appendix B: List of Automatic Checks 52
Appendix C: Operations of the Graphical Interface 53

Abstract

This report documents a MATLAB toolbox built for the course project *Smart Image Processing Toolbox for Real-World Applications* and applied to the project topic **License Plate Recognition Using SVM**. The toolbox has three modules: image enhancement (histogram processing, spatial and frequency-domain filtering), image segmentation with edge detection and morphology, and geometric transformations with interpolation and image registration. A graphical interface exposes 48 operations. All algorithms are implemented from scratch in base MATLAB: no Image Processing Toolbox or Statistics and Machine Learning Toolbox function is used, including the support vector machine, which is trained by a sequential-minimal-optimisation solver written for this project. The code base has 61 files and 3,406 lines, of which 574 are comments.

The modules are combined into a licence plate recogniser. The plate is located with edge and morphological analysis, the characters are cut out by connected-component analysis, and each character is classified by a one-vs-one RBF support vector machine on histogram-of-oriented-gradients features. No photograph collection was available, so the classifier is trained on characters rendered from system fonts and degraded with the toolbox's own operations, and the complete system is tested on synthetic street scenes drawn in fonts that were never used for training.

On 100 such scenes of medium difficulty the plate is located in 99 % of the images, and 83 % of the plates are read exactly when the letter/digit layout is known (75 % without that hint). **No real photograph has been tested.** These figures therefore show that the pipeline works and where it fails; they are not a prediction of accuracy in the field. The report gives the theory, the MATLAB implementation and the measured results of every module, an error analysis of the recogniser and the limitations of the approach.

Keywords: image processing, histogram equalisation, Canny edge detector, Otsu thresholding, morphology, interpolation, image registration, support vector machine, SMO, HOG, licence plate recognition, MATLAB.

1 Introduction

1.1 Background and motivation

Automatic licence plate recognition (ALPR) reads the registration number of a vehicle from a camera image. It is used for toll collection, parking management, access control and traffic-law enforcement. A classical ALPR system has three stages: *localisation* of the plate in the scene, *segmentation* of the plate into single characters, and *recognition* of every character. Each stage relies on a different part of the digital image processing curriculum, which makes ALPR a good vehicle for a toolbox project: contrast enhancement and smoothing prepare the image, edge detection, thresholding and morphology find and cut the plate, geometric transformations correct its orientation, and a classifier turns the character images into text.

Support vector machines (SVMs) are a natural classifier for the last stage. They work well with a few hundred training examples per class, are less prone to over-fitting than large neural networks on small data sets, and combine well with hand-designed features such as histograms of oriented gradients (HOG).

1.2 Project objectives

The assignment (Project.docx) asks the group to develop a custom image-processing toolbox in MATLAB that integrates several techniques and to apply it to a practical problem chosen from a list of topics. Our topic is *License Plate Recognition Using SVM*. The deliverables are:

- **Module 1 – image enhancement:** histogram equalisation and matching, brightness and contrast transformations, smoothing and sharpening filters, and Fourier-domain low-pass, high-pass and band-pass filters.
- **Module 2 – segmentation and edge detection:** Sobel, Prewitt and Canny edge detectors with a comparison, Otsu thresholding, region growing, colour segmentation in RGB/HSV, and morphological operations including shape-based segmentation.
- **Module 3 – geometry and interpolation:** translation, scaling, rotation and affine transformations with forward and reverse mapping, radial distortion correction, nearest-neighbour, bilinear and bicubic interpolation, and (optionally) image registration.
- **A MATLAB graphical user interface** that lets a user load an image, choose a technique and visualise the result, with every module integrated.
- **This report** and a group presentation.

1.3 Scope and constraints

The development computer has base MATLAB only: neither the Image Processing Toolbox nor the Statistics and Machine Learning Toolbox is installed. Functions such as `imfilter`, `edge`, `bwlabel`, `medfilt2`, `graythresh`, `imrotate` or `fitcsvm` are therefore unavailable. A few basic image functions do ship with base MATLAB (for example `imresize`, `rgb2gray` and `imshow`), but the toolbox does not use them either (it has its own `imResize` and `toGray`). We treated this as part of the task, since the assignment asks us to implement the techniques: every operation, the SVM trainer and the feature extractor are written from scratch and checked against reference calculations (Chapter 8). Apart from matrix arithmetic, the functions used are core MATLAB ones such as `conv2`, `fft2`, `accumarray`, `rgb2hsv`, `imread` and the figure and user-interface functions.

Three further constraints shape the results and should be kept in mind when reading them:

- **No photograph data set.** Training characters are rendered from fonts installed on the computer and test scenes are synthetic. Conclusions about real-world accuracy are limited accordingly.
- **Latin characters only.** The recogniser distinguishes the 36 classes 0–9 and A–Z. Other scripts need their own training characters.
- **One plate per image,** roughly frontal, with dark characters on a light plate (a bright-on-dark option exists but was only exercised in unit tests).

1.4 Team structure and responsibilities

The assignment divides the work between three students. Table 1 summarises the split and where each part of the code lives.

Table 1. Division of work between the team members.

Student	Part	Main code	Member
Student 1	Module 1: enhancement and filtering (spatial and frequency domain)	`toolbox/module1_enhancement` (8 files, 298 lines)	Name
Student 2	Module 2: segmentation, edge detection, morphology	`toolbox/module2_segmentation` (13 files, 515 lines)	Name
Student 3	Module 3: geometric transformations and interpolation; integration and testing	`toolbox/module3_geometry` (8 files, 445 lines), GUI, `run_tests.m`	Name
Shared	SVM, plate recogniser, data synthesis, report and presentation	`toolbox/svm`, `toolbox/lpr`	Who worked on what

Describe how the team coordinated: how tasks were agreed, how code was exchanged and reviewed, and any change to the split above.

1.5 Organisation of the report

Chapter 2 describes the architecture and the software structure. Chapters 3 to 5 present Modules 1 to 3; each has the same layout: objectives, theory, MATLAB implementation, measured results and discussion. Chapter 6 describes the SVM and the licence plate recogniser, with training, evaluation and error analysis. Chapter 7 describes the graphical interface, Chapter 8 the testing, and Chapter 9 discusses strengths, limitations and real-world applications. Chapter 10 lists the challenges met during development and Chapter 11 concludes. All figures and numbers were generated by a script (`report/make_report_assets.m`) from the code in the project, not typed in by hand.

2 System overview

2.1 Architecture

Figure 1 shows how the three modules and the SVM form the licence plate recogniser. Colours mark which module a block belongs to. The same Module 1–3 operations are also used outside the recogniser: Module 3 warps the synthetic training characters and the synthetic test scenes, Module 1 adds blur and noise to them, and Module 2 binarises them exactly as the recogniser binarises a real plate.

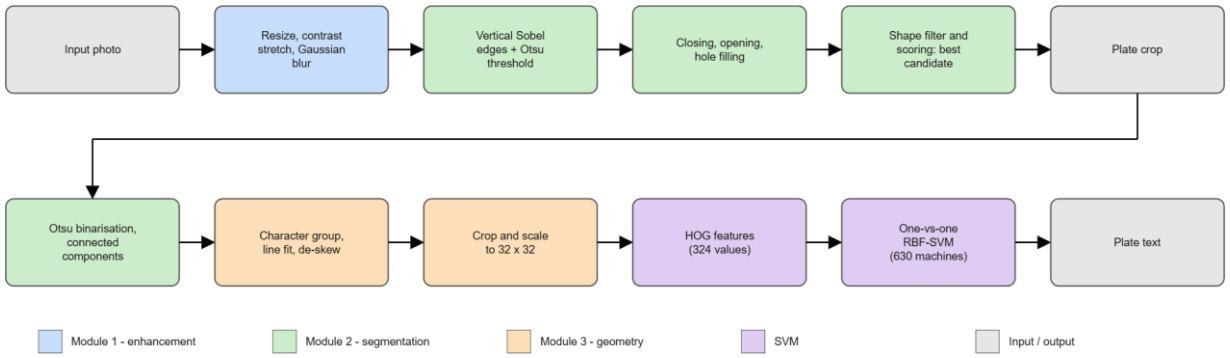


Figure 1. Block diagram of the recogniser. Blue: Module 1, green: Module 2, orange: Module 3, purple: SVM.

Table 2 lists the toolbox functions that each stage calls, which is the practical link between the three student modules and the project topic.

Table 2. Toolbox functions used by each stage of the recogniser.

Stage	Module 1	Module 2	Module 3
Plate detection	`grayTransform` (stretch), `smoothFilter`	`otsuThreshold`, `morphOp`, `fillHoles`, `labelComponents`, `componentProps`, `colorSegment`	`imResize`
Character segmentation	`smoothFilter`	`otsuThreshold`, `labelComponents`, `componentProps`	`geometricTransforms`, `warpImage`, `imResize`
Training-data synthesis	`spatialFilter` (blur), noise	`otsuThreshold`, `labelComponents`	`geometricTransforms` (shear, rotate), `imResize`
Test-scene synthesis	`spatialFilter` (blur, noise)	–	`warpImage`, `imResize`

2.2 Software structure

The project folder contains 61 MATLAB files (3,406 lines, 574 of them comment lines). Table 3 shows the folders. Every function starts with a help header describing inputs, outputs and the algorithm, so `help histEqualize` works for all functions.

Table 3. Folder structure and size of the code base.

Folder	Purpose	Files	Lines
`toolbox/common`	Shared helpers: type conversion, padding, filtering, display, quality metrics	7	126
`toolbox/module1_enhancement`	Module 1: histogram, gray-level, spatial and frequency-domain operations	8	298
`toolbox/module2_segmentation`	Module 2: edges, thresholds, region growing, colour, morphology, labelling	13	515

Folder	Purpose	Files	Lines
`toolbox/module3_geometry`	Module 3: warping, interpolation, distortion, registration	8	445
`toolbox/svm`	Kernel matrix, SMO trainer, multi-class SVM, HOG features	5	257
`toolbox/lpr`	Plate detection, character segmentation, recogniser, data synthesis, evaluation	13	943
`root / toolbox`	GUI, operation table, demos, training script, test suite	7	822

2.3 Design conventions

- **One image format.** Every function converts its input with `toDouble` to double precision in [0, 1] and returns the same range, so operations can be chained freely.
- **Reverse mapping.** Geometric operations compute, for every output pixel, where it comes from in the input and interpolate there; this never leaves holes (Section 5.2).
- **Vectorised code.** Loops over pixels are avoided. Median filtering stacks shifted copies of the image, rectangular morphology is separable, and connected-component labelling works on pixel runs rather than pixels.
- **Deterministic experiments.** Every experiment seeds the random number generator, so the reported numbers can be reproduced exactly (Appendix A).
- **One operation table.** The GUI builds all its controls from a single table (`toolboxOperations.m`), and the tests use the same table, so a new operation needs one new line.

2.4 Development and test environment

Development and all measurements used MATLAB 26.1.0.3312084 on Windows 11 with base MATLAB only. The glyph renderer found 21 font variants on the computer (17 used for training and 4 held out for testing). Timings in this report were measured on the same computer and are given to show orders of magnitude, not as benchmarks.

3 Module 1: Image Enhancement

Responsible: Student 1 name

3.1 Objectives

- Histogram processing: equalisation and matching, and gray-level transformations to adjust brightness and contrast.
- Spatial filtering: smoothing filters (mean, Gaussian, median) for noise reduction and sharpening filters (Laplacian, high-pass, unsharp masking).
- Frequency-domain filtering: Fourier-based low-pass, high-pass and band-pass filters.
- For every technique, a function with documented inputs and outputs and a display of the original and enhanced image for comparison.

3.2 Theory

3.2.1 Histogram processing

For an image with L gray levels r_k, the normalised histogram $p(r_k) = n_k/(MN)$ is the fraction of pixels at level k, and the cumulative distribution function (CDF) is

$$cdf(r_k) = \sum_{j=0}^{k} p(r_j) \tag{1}$$

Histogram equalisation maps every level through the CDF so that the output histogram is approximately flat. With the smallest non-zero CDF value cdf_{min} the transfer function is

$$s_k = \frac{cdf(r_k) - cdf_{min}}{1 - cdf_{min}} \tag{2}$$

Histogram matching (specification) generalises this: with T the CDF of the input and G the CDF of the reference image, each input level r is mapped to the reference level z for which G(z) is closest to T(r) from above,

$$z = G^{-1}(T(r)) \tag{3}$$

which is computed with a monotone search over the two CDFs. For colour images only the V channel of the HSV representation is changed, so hue and saturation, and therefore the colours, are preserved.

3.2.2 Gray-level transformations

Point operations s = T(r) act on every pixel independently. The toolbox implements the linear brightness/contrast transform with gain a and offset b, the gamma transform, the logarithmic transform and a percentile-based contrast stretch:

$$s = a(r - 0.5) + 0.5 + b \tag{4}$$

$$s = r^{\gamma} \tag{5}$$

$$s = \frac{log(1+ar)}{log(1+a)} \tag{6}$$

Gamma below 1 brightens dark regions, gamma above 1 darkens bright ones. The stretch maps the lower and upper percentile (by default 1 % and 99 %) to 0 and 1 and clips the rest, which makes it robust against a few extreme pixels.

3.2.3 Spatial filtering

A spatial filter replaces each pixel by a weighted sum of its neighbourhood, with weights w(s, t) given by a kernel (correlation):

$$g(x,y) = \sum_s \sum_t w(s,t) f(x+s, y+t) \tag{7}$$

The *mean filter* uses equal weights $1/k^2$ and the *Gaussian filter* uses $w(s,t) \propto \exp(-(s^2+t^2)/(2\sigma^2))$, normalised to sum to one; both are linear and blur edges, the Gaussian less abruptly. The *median filter* is non-linear: it replaces the pixel by the median of its k × k neighbourhood, which removes isolated outliers (salt-and-pepper noise) without smearing them over the neighbourhood.

Sharpening adds back a measure of local intensity change. The *Laplacian* (here the eight-neighbour version, whose centre coefficient is negative) is the sum of the eight neighbours minus eight times the centre, and the sharpened image is

$$g = f - c(\nabla^2 f) \tag{8}$$

High-pass sharpening adds a zero-sum 3 × 3 high-pass response, and *unsharp masking* (high-boost filtering) subtracts a Gaussian-blurred copy,

$$g = f + k(f - f_{blur}) \tag{9}$$

with an amount k that controls the strength. All sharpening filters amplify noise together with edges, so they should follow a smoothing step.

3.2.4 Frequency-domain filtering

Filtering in the Fourier domain multiplies the centred spectrum F(u, v) of the image by a transfer function H(u, v) and transforms back:

$$g = Re\{\mathcal{F}^{-1}[H(u,v)F(u,v)]\} \tag{10}$$

Let D(u, v) be the distance of the frequency (u, v) from the centre of the spectrum and D_0 the cut-off radius. The three low-pass shapes are

$$H_{ideal} = \{1 \; if \; D \le D_0, \; 0 \; otherwise\} \tag{11}$$

$$H_{Butterworth} = \frac{1}{1+\left[\frac{D}{D_0}\right]^{2n}} \tag{12}$$

$$H_{Gauss} = e^{-\frac{D^2}{2{D_0}^2}} \tag{13}$$

The ideal filter cuts sharply and causes ringing (ripples around edges); the Butterworth filter of order n = 2 has a smooth transition; the Gaussian filter has no ringing at all. The corresponding high-pass filter is $H_{hp} = 1 - H_{lp}$, and a band-pass filter is the difference of two low-pass filters with cut-offs D_{hi} and D_{lo}. The implementation zero-pads the image to twice its size to avoid wrap-around of the circular convolution, expresses the cut-off as a fraction of the largest usable frequency radius, and keeps the DC term for high-pass and band-pass filters so that the mean brightness is unchanged.

3.2.5 Noise models and quality measures

To evaluate denoising, known noise is added: zero-mean *Gaussian* noise with standard deviation σ, *salt-and-pepper* noise that sets a fraction of the pixels to black or white, and multiplicative *speckle* noise. Quality is measured by the mean squared error against the clean image and the peak signal-to-noise ratio, which for images in [0, 1] is

$$PSNR = 10 log_{10}\left(\frac{1}{MSE}\right) \tag{14}$$

Higher PSNR means a result closer to the clean image.

3.3 MATLAB implementation

Table 4 lists the functions of Module 1 together with the shared helpers they use (`toDouble`, `padImage`, `spatialFilter`, `gaussianKernel`, `imageMetrics`). The descriptions are taken from the help headers in the code.

Table 4. Functions of Module 1 (folder toolbox/module1_enhancement).

Function	Purpose
`J = addNoise(I, type, amount)`	Add synthetic noise so the denoising filters can be evaluated.
`fig = compareEnhancement(I, ref)`	Show the original image next to every Module 1 result.
`[J, H, spectrum] = freqFilter(I, type, cutoff, shape, order)`	Fourier-domain filtering (Module 1 - frequency domain).
`J = grayTransform(I, method, p1, p2)`	Point (gray-level) transformations for brightness/contrast.
`J = histEqualize(I, nLevels)`	Histogram equalisation (Module 1 - histogram processing).
`J = histMatch(I, ref, nLevels)`	Histogram matching / specification (Module 1).
`[J, detail] = sharpenFilter(I, type, amount)`	Image sharpening (Module 1 - spatial filtering).
`J = smoothFilter(I, type, k, sigma)`	Spatial smoothing filters for noise reduction (Module 1).

Implementation notes:

- **Histograms** are computed with `accumarray`: the quantised image is used as the index into a 256-bin counter, so no loop over pixels is needed. Equalisation and matching are then table look-ups.
- **Spatial filters** are correlations with an odd-sized kernel through `conv2` on a border-padded image (`padImage`, replicate/symmetric/zero), so the output has the size of the input and no black frame appears.
- **The median filter** stacks the k^2 shifted copies of the image into a three-dimensional array and takes the median along the third dimension, which is fully vectorised.
- **Frequency filtering** builds H(u, v) on the padded frequency grid, applies `fftshift` and `ifftshift` around the multiplication, crops the result, and also returns H and the log-magnitude spectrum for display.
- **Colour images** are handled per channel by the spatial and frequency filters and through the V channel by the histogram operations.

3.4 Results

All Module 1 experiments use one synthetic street scene (a car with a licence plate, 640 × 480 pixels) so that the images relate to the project topic. Figure 2 shows the original and the result of one setting of each technique, as produced by `compareEnhancement`.

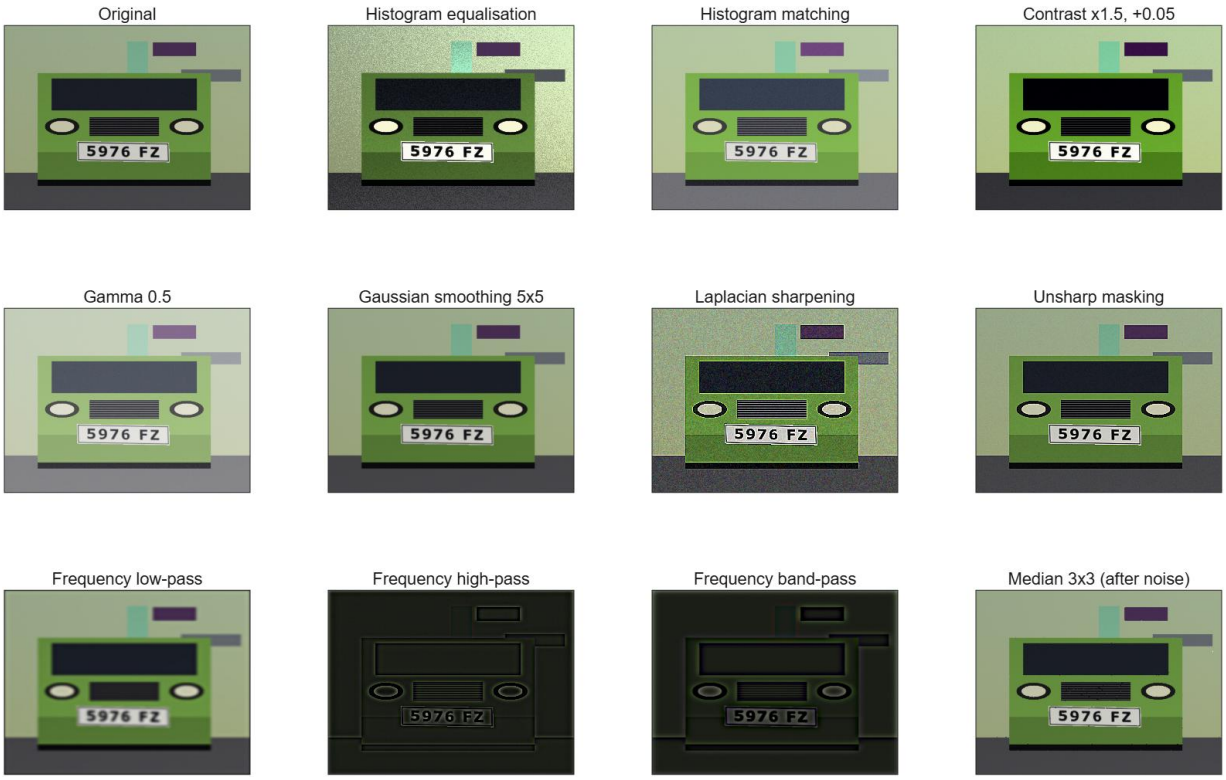


Figure 2. Original image and one result of each Module 1 technique (output of compareEnhancement).

3.4.1 Histogram processing

A low-contrast copy of the scene (gray values compressed into [0.30, 0.55]) was equalised and matched to the original scene. Figure 3 shows the images and their histograms; Table 5 gives the statistics.

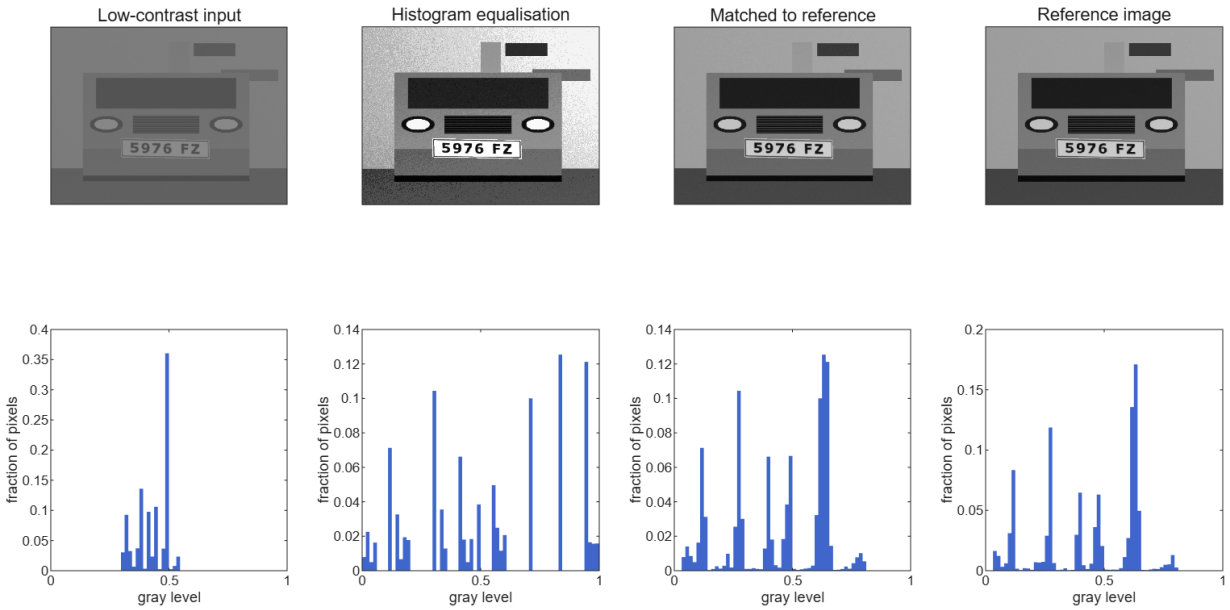


Figure 3. Histogram equalisation and histogram matching of a low-contrast image, with the gray-level histograms below.

Table 5. Statistics of the histogram experiment.

Image	Mean	Std. deviation	Entropy (bits)
Low-contrast input	0.43	0.06	4.42
After equalisation	–	0.30	4.34
After matching to the reference	0.45	0.21	–

Image	Mean	Std. deviation	Entropy (bits)
Reference (well exposed)	0.44	0.20	–

3.4.2 Noise reduction

Gaussian noise (σ = 0.08) and salt-and-pepper noise (10 % of the pixels) were added to the scene and removed with four filters each. Figure 4 shows a close-up of the licence plate area; Table 6 gives the PSNR of the whole image against the clean scene.

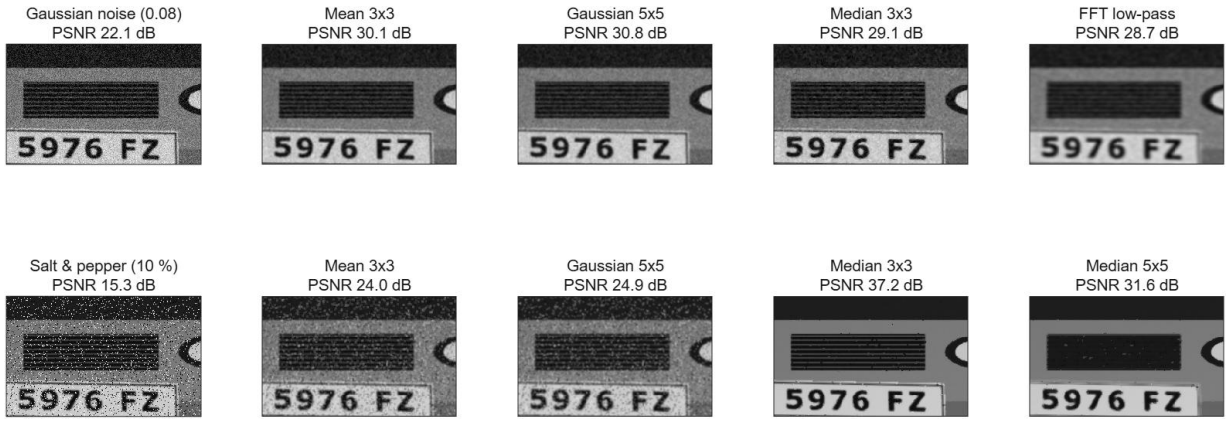


Figure 4. Noise reduction (close-up of the grille and plate). Top: Gaussian noise. Bottom: salt-and-pepper noise.

Table 6. PSNR (dB) of the denoised image against the clean image. The FFT low-pass is a Butterworth filter of order 2 with cut-off 0.25. Best value per row in bold.

Noise	Noisy	Mean 3×3	Gaussian 5×5	Median 3×3	Median 5×5	FFT low-pass
Gaussian, σ = 0.08	22.1	30.1	**30.8**	29.1	–	28.7
Salt and pepper, 10 %	15.3	24.0	24.9	**37.2**	31.6	–

3.4.3 Sharpening

Sharpening was compared by the standard deviation of the Laplacian response, which grows when edges become steeper: the original scene scores 0.052, Laplacian sharpening (amount 1) 0.303, high-pass sharpening 0.164 and unsharp masking (amount 1.5) 0.119. These numbers include the camera noise of the scene, which every sharpening filter amplifies as well; the visual comparison in the overview figure shows the same: the Laplacian result has the crispest characters and the most visible noise.

3.4.4 Frequency-domain filtering

Figure 5 shows the spectrum of the scene, the three low-pass transfer functions and the filtered results (close-up). Table 7 compares the shapes quantitatively, both as a blurring operation (PSNR of the low-pass result against the original, cut-off 0.10) and as a denoiser for the Gaussian-noise image.

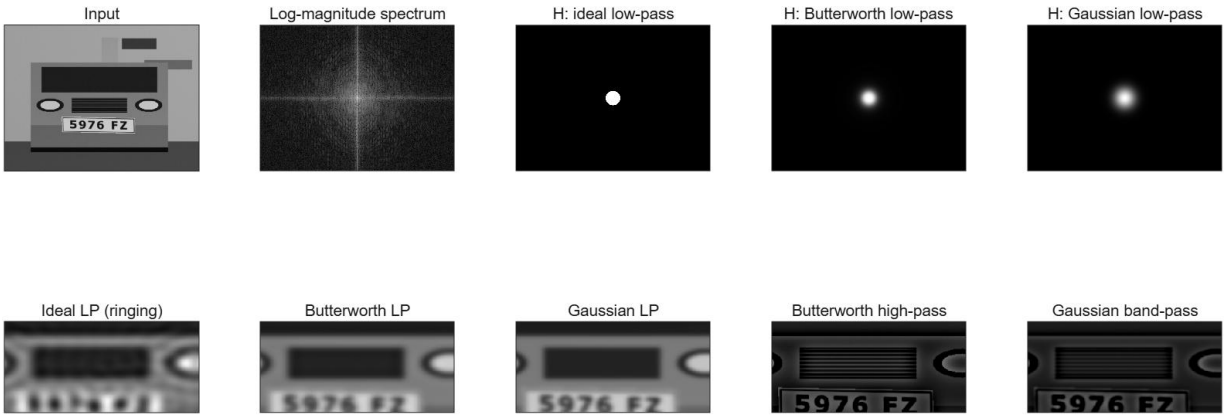


Figure 5. Frequency-domain filtering. Top: input, centred log spectrum and low-pass transfer functions. Bottom: results (close-up) of ideal, Butterworth and Gaussian low-pass, Butterworth high-pass and Gaussian band-pass.

Table 7. PSNR (dB) of low-pass filtering. The noisy input has 22.1 dB.

Filter shape	Blur: PSNR vs. original (cut-off 0.10)	Denoise cut-off 0.15	Denoise cut-off 0.25	Denoise cut-off 0.35
Ideal	23.3	25.0	27.6	28.8
Butterworth	24.2	26.1	28.7	30.0
Gaussian	24.9	26.8	29.3	30.1

3.5 Discussion

- **Histogram processing works as designed.** Equalisation raised the standard deviation of the low-contrast image from 0.06 to 0.30. Its entropy fell slightly (4.42 to 4.34 bits), which is expected for a discrete many-to-one mapping: equalisation spreads the levels but cannot add information. Matching to the reference produced a mean of 0.45 and a standard deviation of 0.21 against 0.44 and 0.20 for the reference.
- **The right denoising filter depends on the noise.** For salt-and-pepper noise the 3 × 3 median filter improved the PSNR from 15.3 to 37.2 dB, about 12.4 dB better than the Gaussian and 13.3 dB better than the mean filter, which smear the outliers over their neighbours. A larger 5 × 5 median (31.6 dB) removes thin structures such as the grille slats, so the smallest window that removes the noise is preferable. For Gaussian noise the linear filters win: the Gaussian 5 × 5 (30.8 dB) is better than the median (29.1 dB).
- **Filter shape matters in the frequency domain.** At the same cut-off the ideal low-pass filter produced the lowest PSNR (23.3 dB) and visible ringing around the characters, the Butterworth filter 24.2 dB, and the Gaussian filter the highest (24.9 dB) without ringing. The same order held for denoising at all three cut-offs, for example 29.3, 28.7 and 27.6 dB at cut-off 0.25. Higher cut-offs gave better PSNR here because the scene is dominated by low frequencies while the noise is spread over all frequencies.
- **Sharpening needs clean input.** Every sharpening filter increased the edge energy, but also the visible noise. For this reason the plate detector of Chapter 6 smooths with a Gaussian filter and does not sharpen.
- **Limits.** The experiments use one scene and one noise realisation per noise type, so the PSNR differences of a few tenths of a dB (for example between the mean and Gaussian filters on Gaussian noise) should not be over-interpreted; the large differences (median versus linear filters on impulse noise) are robust.

4 Module 2: Segmentation and Edge Detection

Responsible: Student 2 name

4.1 Objectives

- Edge detection with the Sobel, Prewitt and Canny operators, and a function that compares them.
- Segmentation by thresholding (Otsu's method), region growing, and colour segmentation in RGB and HSV space.
- Morphological operations (dilation, erosion, opening, closing) for refining segmented images, and shape-based segmentation.
- Visual and quantitative comparisons between the techniques.

4.2 Theory

4.2.1 Gradient-based edge detection

An edge is a place where the intensity changes quickly, so it can be found from the image gradient. The Sobel and Prewitt operators approximate the horizontal derivative G_x with a 3 × 3 kernel that differentiates in one direction and smooths in the other; the vertical derivative G_y uses the transposed kernel. Sobel weights the central row by 2, Prewitt weights all rows equally:

Table 8. Derivative kernels of the Sobel and Prewitt operators.

Operator	Kernel for G_x	Kernel for G_y
Sobel	[−1 0 1; −2 0 2; −1 0 1]	[−1 −2 −1; 0 0 0; 1 2 1]
Prewitt	[−1 0 1; −1 0 1; −1 0 1]	[−1 −1 −1; 0 0 0; 1 1 1]

The gradient magnitude and direction follow from the two responses, and a pixel is marked as an edge when the magnitude exceeds a threshold:

$$|\nabla f| = \sqrt{{G_x}^2 + {G_y}^2} \qquad \theta = atan2(G_y, G_x) \tag{15}$$

The toolbox takes the threshold as a fraction of the largest magnitude; if none is given it is chosen automatically with Otsu's method applied to the magnitude image.

The *Canny detector* improves on plain thresholding with five steps: (1) Gaussian smoothing with standard deviation σ (default 1.4) to suppress noise; (2) Sobel gradient magnitude and direction; (3) *non-maximum suppression*, which keeps a pixel only if its magnitude is not smaller than that of its two neighbours along the gradient direction (quantised to 0°, 45°, 90° and 135°), thinning edges to one pixel; (4) a *double threshold* with a high value t_h and a low value $t_l = 0.4\ t_h$; (5) *hysteresis*: weak edge pixels (above t_l) are kept only if their connected component contains a strong pixel (above t_h). The result is thin, connected edges with few isolated noise pixels.

4.2.2 Thresholding

Otsu's method chooses the global threshold k that maximises the between-class variance of the two pixel classes (background and foreground) it creates. With the class probability ω(k), the cumulative mean μ(k) and the global mean μ_T of the normalised histogram:

$$\sigma_B{}^2(k) = \frac{[\mu_T\omega(k)-\mu(k)]^2}{\omega(k)(1-\omega(k))} \tag{16}$$

The threshold k* is the maximiser. The function also returns the separability measure $\eta = \sigma_B^2(k^*)/\sigma_T^2 \in [0, 1]$, which is close to 1 for a clearly bimodal histogram. A global threshold fails when the illumination varies across the image, so an *adaptive threshold* compares each pixel with the mean of its own window, foreground when $f(x, y) < \text{mean}_{\text{window}}(x, y) - C$ (dark objects) or $f > \text{mean}_{\text{window}} + C$ (bright objects).

4.2.3 Region growing and colour segmentation

Region growing starts from one or more seed pixels, computes a reference intensity m (the mean of the 3 × 3 neighbourhoods around the seeds) and repeatedly adds neighbouring pixels that satisfy $|f(x, y) - m| \le \tau$, until no pixel is added. Implemented as dilation restricted to the candidate mask, each iteration grows the region by one pixel layer.

Colour segmentation selects the pixels whose colour lies in a range. In HSV space, where hue H separates colour from brightness V and saturation S, a range such as "yellow" is a simple box; in RGB space the box is given directly by the three channels. The toolbox provides box presets for white, yellow, blue, red, green and black (for example white: $S \le 0.25$ and $V \ge 0.62$; yellow: $0.09 \le H \le 0.20$, $S \ge 0.35$, $V \ge 0.40$) and accepts custom ranges. The hue range may wrap through 0, which is needed for red.

4.2.4 Morphological operations

Morphology probes an image with a small structuring element (SE) B. For a binary or gray-scale image A the flat dilation and erosion are the maximum and minimum over the SE:

$$(A \oplus B)(x) = \max_{b\in B} A(x-b) \qquad (A \ominus B)(x) = \min_{b\in B} A(x+b) \tag{17}$$

Opening (erosion then dilation) removes objects smaller than the SE, *closing* (dilation then erosion) fills gaps and holes smaller than the SE, and the *morphological gradient* (dilation minus erosion) outlines objects. Top-hat and black-hat (image minus opening, closing minus image) extract small bright or dark details. Shape-based segmentation combines morphology with *connected-component analysis*: after labelling the components, only those whose area, bounding-box aspect ratio and extent (area divided by bounding-box area) match the expected shape are kept; this is how the plate detector selects plate-like blobs (Chapter 6).

4.3 MATLAB implementation

Table 9. Functions of Module 2 (folder toolbox/module2_segmentation).

Function	Purpose
`BW = adaptiveThreshold(I, win, C, polarity)`	Local-mean thresholding, robust against uneven lighting.
`BW = colorSegment(I, space, spec)`	Colour-based segmentation in the RGB or HSV colour space.
`fig = compareEdgeDetectors(I)`	Side-by-side comparison of Sobel, Prewitt and Canny.
`fig = compareSegmentation(I, seed)`	Compare thresholding, region growing, colour and edge+morphology segmentation.
`props = componentProps(L, n)`	Area, bounding box and centroid of every labelled component.
`[E, mag, ang] = edgeDetect(I, method, thresh, sigma)`	Sobel, Prewitt and Canny edge detectors (Module 2).
`BW = fillHoles(BW)`	Fill background regions that are completely surrounded by foreground.
`[BW, keep, props, L] = filterComponents(BW, varargin)`	Shape-based segmentation: keep only components matching size/shape rules.
`[L, n] = labelComponents(BW, conn)`	Connected-component labelling of a binary image (like bwlabel).
`SE = makeSE(type, sz, angle)`	Create a flat structuring element (logical matrix).

Function	Purpose
`J = morphOp(I, op, SE)`	Flat morphological operations for binary and gray-scale images.
`[level, BW, eta] = otsuThreshold(I)`	Otsu's global threshold (maximises between-class variance).
`[BW, nIter] = regionGrow(I, seed, tol, conn)`	Region-growing segmentation from one or more seed points.

Implementation notes:

- **Connected components without <<bwlabel>>.** `labelComponents` first compresses each image row into runs of foreground pixels, then merges runs that overlap in adjacent rows with a union–find structure (4- or 8-connectivity), and finally paints the labels. The cost depends on the number of runs, not pixels, which makes labelling practical in interpreted MATLAB. It is checked against a flood-fill reference on random images (Chapter 8).
- **Morphology** uses maximum/minimum filters over the SE offsets. For rectangular SEs the filter is separable (a vertical pass, then a horizontal pass), which turns a 3 × 60 rectangle from 180 to 63 shifts. Padding with $-\infty$ (dilation) or $+\infty$ (erosion) keeps the borders from being eroded or dilated artificially, and the dilation uses the reflected SE so even-sized elements are handled correctly. The same code serves binary and gray-scale images.
- **Canny hysteresis** reuses `labelComponents`: the weak-edge map is labelled and the components that contain at least one strong pixel are kept.
- **Region growing** is written as iterated 3 × 3 dilation (`conv2`) of the region masked by the candidate set, which avoids a queue and loops.
- **<<fillHoles>>** labels the background with 4-connectivity; background components that do not touch the image border are holes and are filled.
- **Structuring elements** (`makeSE`) can be rectangles, squares, discs, diamonds, crosses and lines at any angle.

4.4 Results

4.4.1 Edge detection

Figure 6 compares the three detectors on the synthetic scene (output of `compareEdgeDetectors`). To measure them, a test image with three shapes of known outline was corrupted with Gaussian noise of two strengths and each detector was run with its automatic threshold (Sobel and Prewitt: Otsu on the gradient magnitude; Canny: the defaults of Section 4.2). A detected pixel counts as correct when it lies within 2 pixels of the true outline, and an outline pixel counts as found when a detected pixel lies within 2 pixels. Figure 7 shows the results and Table 10 gives precision, recall and the F1 score.

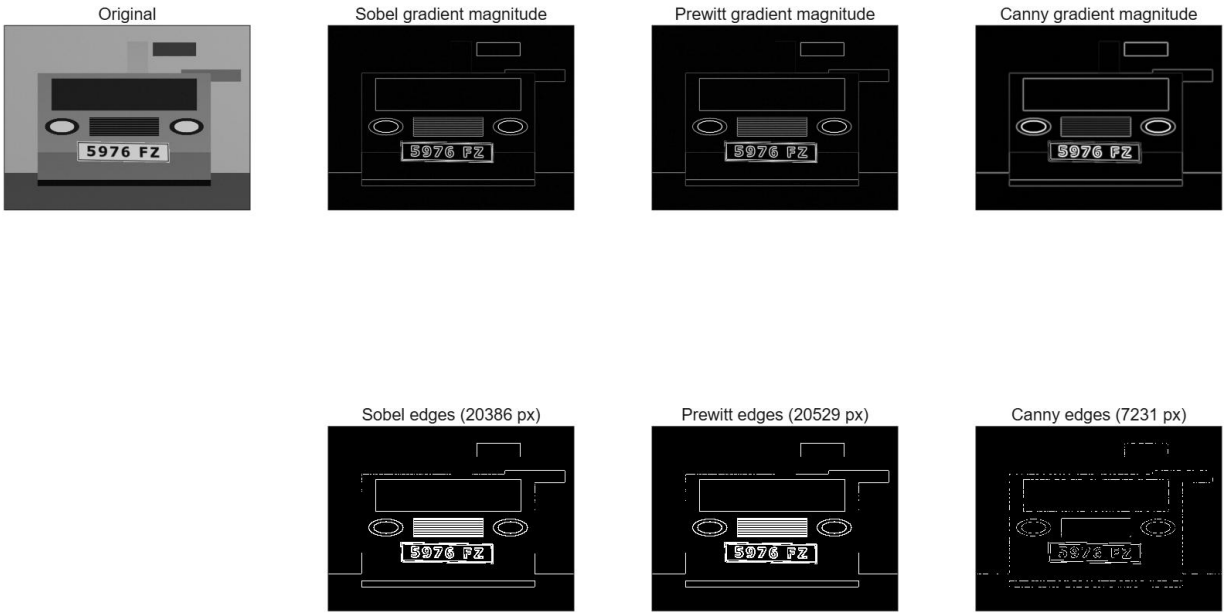


Figure 6. Sobel, Prewitt and Canny on the synthetic scene: gradient magnitudes (top) and edge maps (bottom).

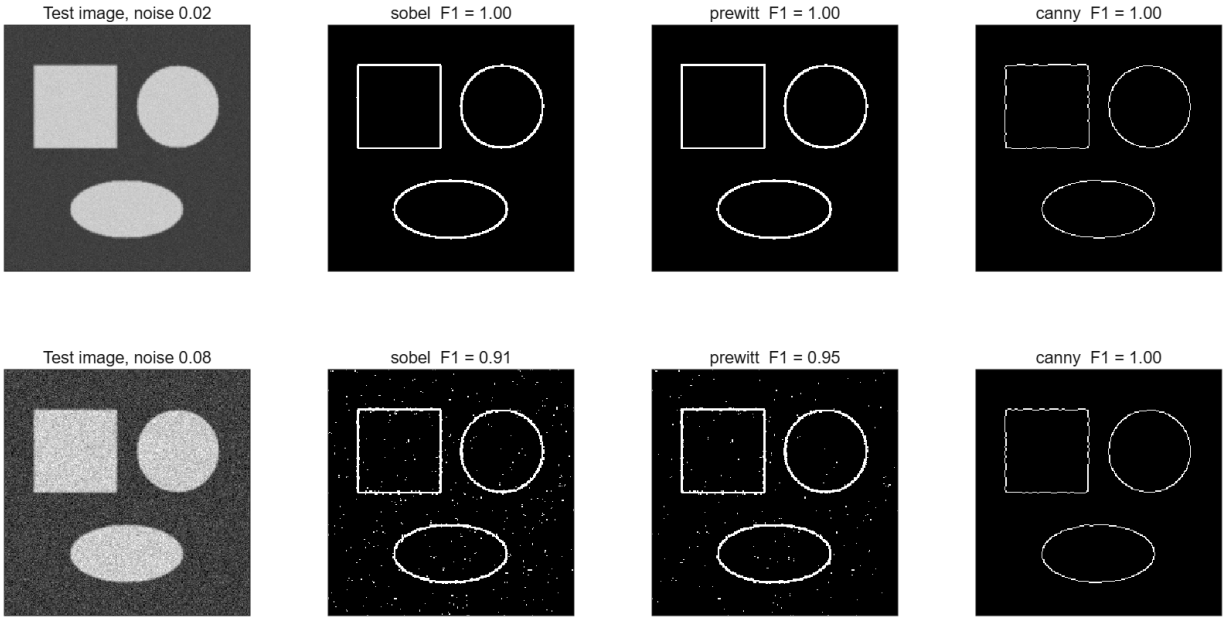


Figure 7. Edge detectors on shapes with known outlines at two noise levels (standard deviation 0.02 and 0.08).

Table 10. Precision (P), recall (R) and F1 score of the edge detectors on the shape test image.

Detector	P (σ = 0.02)	R	F1	P (σ = 0.08)	R	F1
Sobel	1.00	1.00	**1.00**	0.83	1.00	**0.91**
Prewitt	1.00	1.00	**1.00**	0.90	1.00	**0.95**
Canny	1.00	1.00	**1.00**	1.00	1.00	**1.00**

Computation time for the Canny detector on a 640 × 480 image was 69 ms; labelling its 7,231 edge pixels took 8.0 ms.

4.4.2 Thresholding, region growing and colour

Figure 8 shows the segmentation techniques on the synthetic scene (output of `compareSegmentation`): Otsu and adaptive thresholds, region growing from the image centre, colour segmentation for white, Canny edges, edges closed and filled, and the result of shape cleaning.

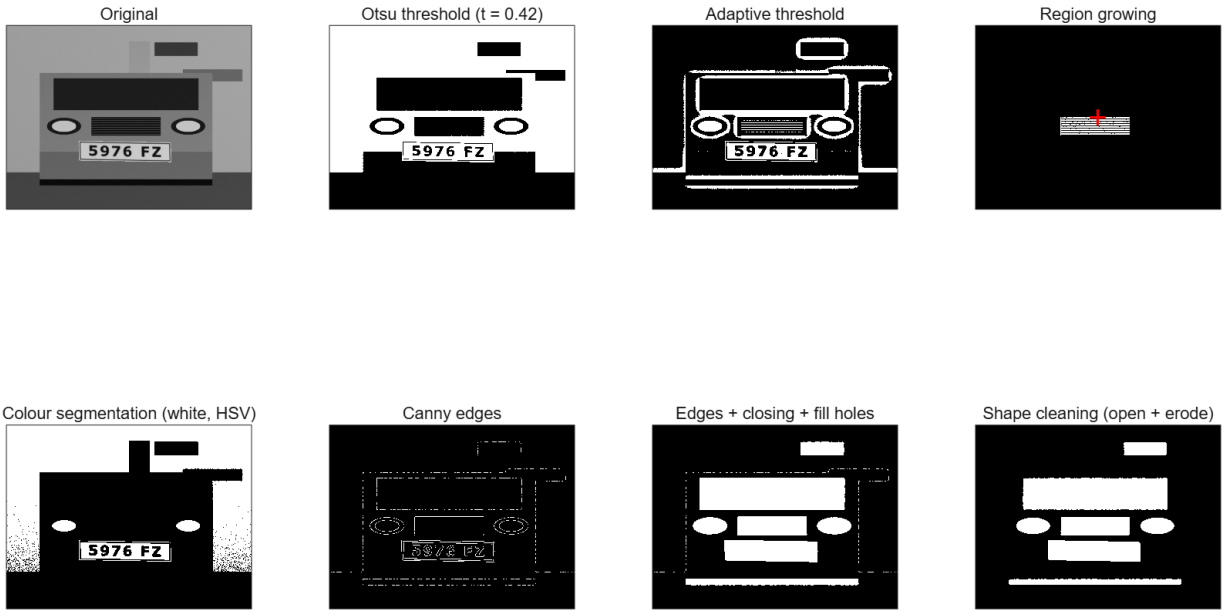


Figure 8. Segmentation techniques on the synthetic scene.

For a quantitative comparison, eight small objects (24–34 pixels) were placed on a dark background under a lighting gradient that darkens the image to 30 % on the left and brightens it to 120 % on the right, with Gaussian noise of 0.03. Every method received the same Gaussian-smoothed image. Figure 9 and Table 11 give the Dice coefficient against the ground truth.

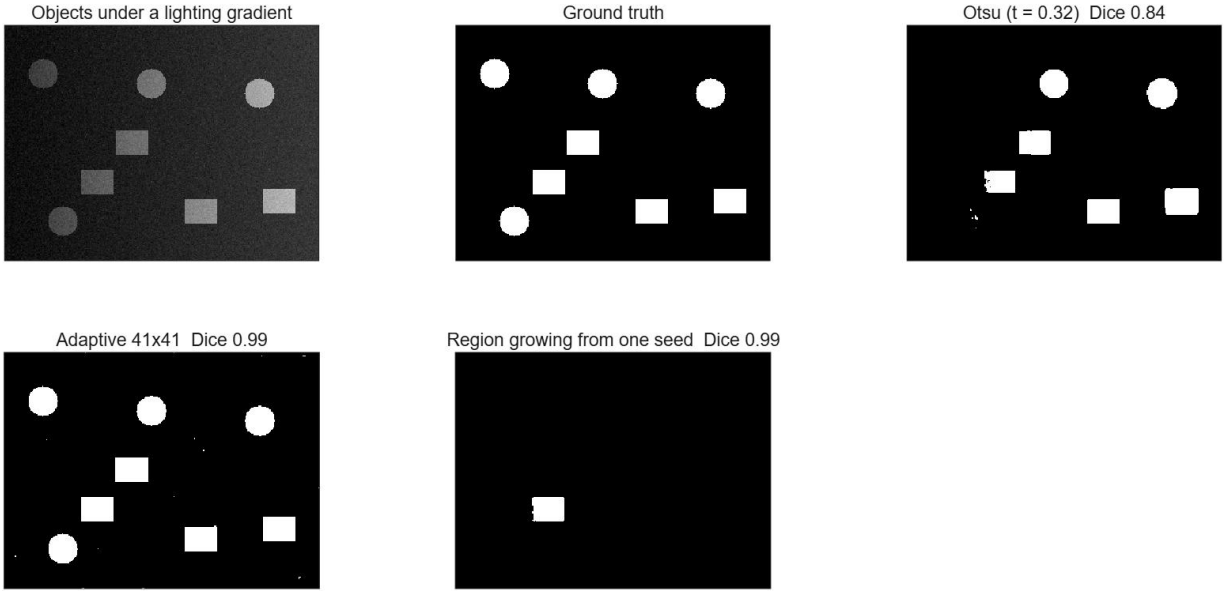


Figure 9. Global, adaptive and region-growing segmentation under uneven lighting.

Table 11. Segmentation quality under uneven lighting.

Method	Dice coefficient
Global Otsu threshold (t = 0.32)	0.84
Adaptive threshold (41 × 41 window, offset 0.03)	**0.99**
Region growing from one seed (tolerance 0.10), measured on that object only	0.99

Region growing was also applied to the licence plate itself: a seed on the white plate background was grown with three tolerances (Figure 10). The segmented area was 1.8 %, 2.0 % and 2.3 % of the image for tolerances

0.03, 0.08, 0.2, and about 215 iterations were needed because the region grows by one pixel layer per iteration.

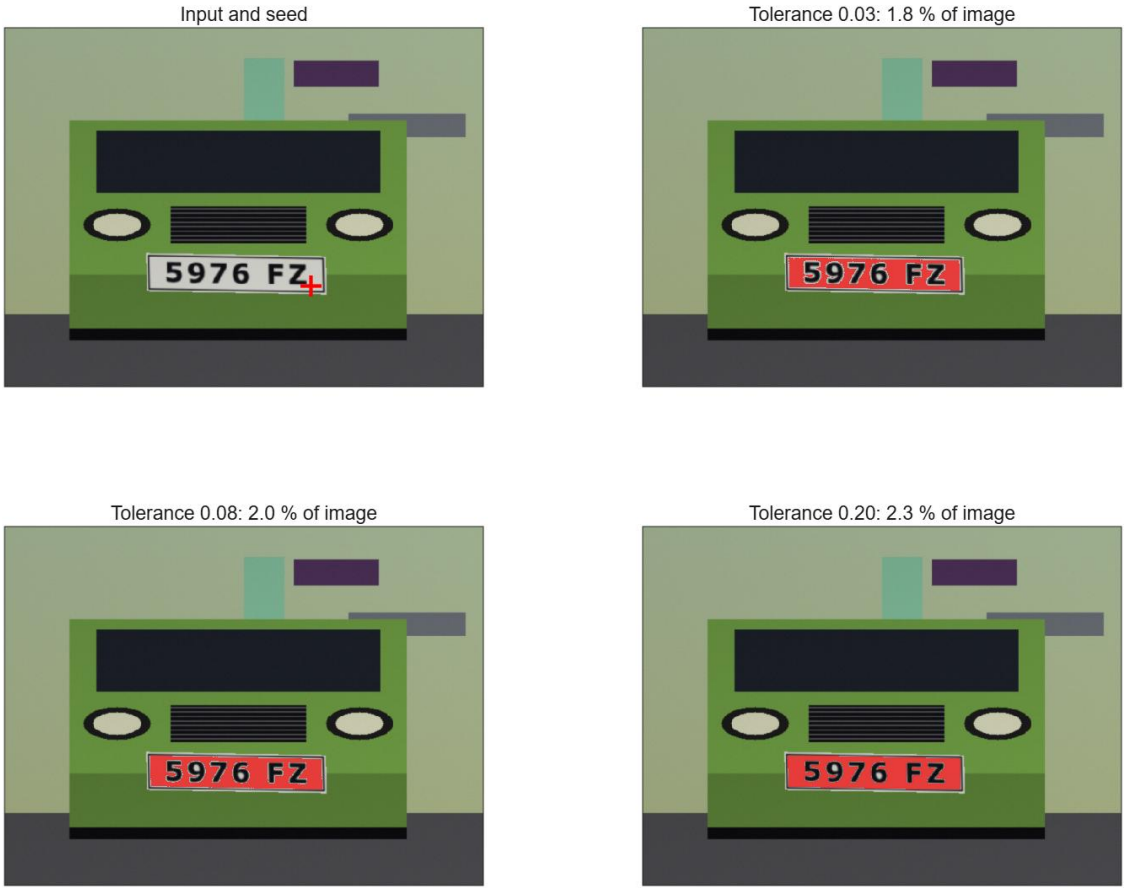


Figure 10. Region growing from a seed on the plate background with tolerance 0.03, 0.08 and 0.20.

4.4.3 Morphological operations

Figure 11 applies the morphological operations to the binarised plate text of the scene after 2 % salt-and-pepper noise had been added. The number of connected components, which should be near that of the clean image, falls from 128 in the noisy image to 37 after a 3 × 3 opening and 20 after opening followed by closing; the clean image has 9.



Figure 11. Morphological operations on binarised plate text (white = foreground).

4.5 Discussion

- **Canny is the most robust detector.** At the low noise level all three detectors reach F1 = 1.00 (within the 2-pixel tolerance). At σ = 0.08 Sobel and Prewitt still find every true edge pixel (recall 1.00) but add many isolated noise pixels (precision 0.83 and 0.90), while Canny, thanks to smoothing and hysteresis, keeps precision and recall at 1.00. The difference between Sobel and Prewitt (F1 0.91 against 0.95) comes from the automatically chosen thresholds, not from a fundamental property of the kernels, and it is from one image and one noise sample, so we do not conclude that one is better. Canny needs more computation (69 ms here).
- **A global threshold fails under uneven lighting.** With a four-fold lighting gradient Otsu's method lost the dim objects on the left (Dice 0.84), whereas the adaptive threshold, which compares each pixel with its own neighbourhood, reached 0.99. This only works when the objects are smaller than the window; large uniform objects would become hollow, which was not tested here. In the recogniser, whose characters are small compared with the plate, the adaptive threshold nevertheless did not improve the results (Section 6.3), so Otsu is the default there.
- **Region growing is precise but needs a seed.** It segmented the chosen object almost perfectly (Dice 0.99), but it cannot find objects by itself and is slow for large regions in this implementation (about 215 iterations for the plate). It is a tool for interactive use or for refining a region found by other means.
- **Morphology removes most, not all, noise.** The 3 × 3 opening reduced the specks considerably but also removed the thin plate frame, and the combined operation still leaves 11 more components than the clean image. A larger structuring element would remove more noise but would also erode thin strokes of the characters; choosing the size is a trade-off between the two.
- **Labelling is fast enough.** The run-based algorithm labelled a Canny edge map of 640 × 480 pixels in 8.0 ms, which is why the detector of Chapter 6 can afford several labelling passes per image.

5 Module 3: Geometric Transformations and Interpolation

Responsible: Student 3 name

5.1 Objectives

- Image translation, scaling (zooming), rotation and general affine transformations.
- Forward and reverse mapping for image warping.
- Correction of radial lens distortion.
- Nearest-neighbour, bilinear and bicubic interpolation for resizing and transformation.
- Optional and advanced: a basic image registration algorithm based on feature matching.
- Integration of the three modules into one toolbox (graphical interface) and testing of the whole.

5.2 Theory

5.2.1 Transformations in homogeneous coordinates

A point (x, y) is written as the column vector $[x\ y\ 1]^T$ so that translations as well as linear maps become matrix products. A transformation T maps an input point to an output point, $[x'\ y'\ 1]^T = T\ [x\ y\ 1]^T$. The elementary transformations are

Table 12. Elementary geometric transformations.

Transformation	Matrix T	Parameters
Translation	$[1\ 0\ t_x;\ 0\ 1\ t_y;\ 0\ 0\ 1]$	shift (t_x, t_y) in pixels
Scaling	$[s_x\ 0\ 0;\ 0\ s_y\ 0;\ 0\ 0\ 1]$	zoom factors s_x, s_y
Rotation	$[\cos\theta\ \sin\theta\ 0;\ -\sin\theta\ \cos\theta\ 0;\ 0\ 0\ 1]$	angle θ (counter-clockwise on screen, y axis points down)
Shear	$[1\ h_x\ 0;\ h_y\ 1\ 0;\ 0\ 0\ 1]$	shear factors h_x, h_y
General affine	$[a\ b\ t_x;\ c\ d\ t_y;\ 0\ 0\ 1]$	six free parameters

Transformations are composed by multiplying their matrices. Scaling and rotation of an image act about the origin, so about the image centre c the operation is $T = T_c \cdot R \cdot T_c^{-1}$, where T_c translates the origin to c.

5.2.2 Forward and reverse mapping

Forward mapping takes every input pixel (x, y), computes where it lands, (x′, y′) = T(x, y), and writes its value there. Because (x′, y′) is rarely an integer position, the value has to be rounded to a pixel, and when the image is enlarged some output pixels receive no value at all (holes) while others receive several. *Reverse mapping* instead visits every output pixel (x′, y′), computes the source position with the inverse transform,

$$(x, y) = T^{-1}(x', y') \tag{18}$$

and interpolates the input image at that position. Every output pixel gets a value, so there are no holes; this is the method used by all geometric functions, while forward mapping is provided to demonstrate the problem.

5.2.3 Interpolation

The source position found by reverse mapping is generally non-integer, so a value has to be estimated from the surrounding pixels. *Nearest-neighbour* interpolation copies the closest pixel: it is the fastest and exact for

integer shifts, but makes edges jagged. *Bilinear* interpolation weights the four surrounding pixels f_{00}, f_{10}, f_{01} and f_{11} with the fractional offsets a = x − ⌊x⌋ and c = y − ⌊y⌋:

$$f(x,y) = (1-a)(1-c)f_{00} + a(1-c)f_{10} + (1-a)cf_{01} + acf_{11} \tag{19}$$

Bicubic interpolation uses the 4 × 4 neighbourhood with the cubic convolution kernel of Keys (parameter α = −0.5), which reproduces linear and quadratic functions exactly and preserves edges better than bilinear interpolation:

$$w(t) = \{(\alpha+2)|t|^3 - (\alpha+3)|t|^2 + 1 \ (|t| \le 1);\ \alpha|t|^3 - 5\alpha|t|^2 + 8\alpha|t| - 4\alpha \ (1 < |t| < 2);\ 0 \ otherwise\} \tag{20}$$

Resizing uses reverse mapping with aligned pixel centres, $x_{in} = (x_{out} - 0.5)/s + 0.5$ for scale s, and clamps the coordinates so that the border pixels are replicated instead of turning black. When an image is shrunk, a Gaussian low-pass filter with σ = 0.5/s is applied first to avoid aliasing.

5.2.4 Radial lens distortion

Wide-angle lenses bend straight lines near the image border. The Brown model describes the distorted position of a point in normalised coordinates (origin at the image centre, radius 1 at the corner):

$$x_d = x_u(1 + k_1 {r_u}^2 + k_2 {r_u}^4) \tag{21}$$

Negative k_1 gives barrel distortion and positive k_1 pincushion distortion. Correcting a photo is reverse mapping: for each pixel of the corrected image the distorted position is computed with the equation above and interpolated. Simulating the distortion (used for testing) needs the inverse of the model, which is obtained by the fixed-point iteration $x_u \leftarrow x_d / (1 + k_1 r_u^2 + k_2 r_u^4)$, repeated 20 times.

5.2.5 Image registration

Registration finds the transformation that aligns a moving image with a fixed image. Two methods are implemented. *Phase correlation* finds a pure translation from the normalised cross-power spectrum of the two images:

$$R = \frac{F_1 F_2^*}{|F_1 F_2^*|} \qquad (\Delta x, \Delta y) = \underset{x,y}{argmax}\, \mathcal{F}^{-1}(R) \tag{22}$$

The *feature-based method* handles rotation, translation and moderate scaling: (1) Harris corners are detected from the structure tensor M of the gradients, R = det M − κ tr²M with κ = 0.04; (2) each corner is described by a 17 × 17 patch normalised to zero mean and unit norm; (3) corners are matched by normalised cross-correlation, keeping only mutual best matches whose score is clearly better than that of the second-best candidate (ratio test); (4) RANSAC repeatedly fits an affine transformation to three random matches and keeps the model with the most inliers (distance below 3 pixels), which is finally refitted to all inliers by least squares; (5) the moving image is warped with reverse mapping.

5.3 MATLAB implementation

Table 13. Functions of Module 3 (folder toolbox/module3_geometry).

Function	Purpose
`fig = compareGeometry(I)`	Show Module 3 results: transforms, interpolation methods and distortion.
`varargout = geometricTransforms(I, op, varargin)`	Translation, scaling, rotation, shear and general affine warps.
`pts = harrisCorners(I, maxPts, sigma)`	Harris corner detector. Returns up to maxPts corners as [x y score].
`J = imResize(I, sz, method, antialias)`	Resize an image with nearest / bilinear / bicubic interpolation.
`J = interpSample(I, X, Y, method, fillVal)`	Sample an image at non-integer coordinates (Module 3 - interpolation).

Function	Purpose
`J = radialDistortion(I, k1, k2, mode, method)`	Apply or correct radial (barrel / pincushion) lens distortion.
`[T, registered, info] = registerImages(fixed, moving, method)`	Align a moving image to a fixed image (Module 3 - optional, advanced).
`J = warpImage(I, T, outSize, method, mapping, fillVal)`	Geometric warp by a 3x3 homogeneous transform (forward or reverse mapping).

Implementation notes:

- **One warping engine.** `warpImage` takes a 3 × 3 matrix, builds the output pixel grid with `meshgrid`, applies `T \ points` to obtain the source positions and calls `interpSample`. `geometricTransforms` only builds the matrix (and the larger canvas for a full rotation), so translation, scaling, rotation, shear and affine maps share the same, tested code.
- **Interpolation by index arithmetic.** `interpSample` works on arrays of coordinates; for the bicubic case it accumulates the 16 weighted neighbours in two nested loops over the kernel offsets, with clamped indices at the border and a small tolerance (10^{-6} pixel) so that floating-point error does not turn border pixels black.
- **Forward mapping** writes values with `accumarray`; the bilinear variant splats each input pixel onto its four output neighbours with bilinear weights and normalises by the accumulated weight. Output pixels that receive no weight stay at the fill value, which is how the holes are measured.
- **Registration** reuses Module 2 for local-maximum detection (`morph0p` dilation) and Module 1 for the Gaussian window, and returns the transformation matrix, the registered image and diagnostics (matches, inliers or shift).

5.4 Results

5.4.1 Geometric transformations

Figure 12 shows the transformations on the synthetic scene (output of `compareGeometry`): translation, scaling, rotation with the same and with an enlarged canvas, shear, forward and reverse mapping of the same scaling, barrel distortion with its correction, and two enlarged crops that expose the difference between nearest-neighbour and bicubic interpolation.

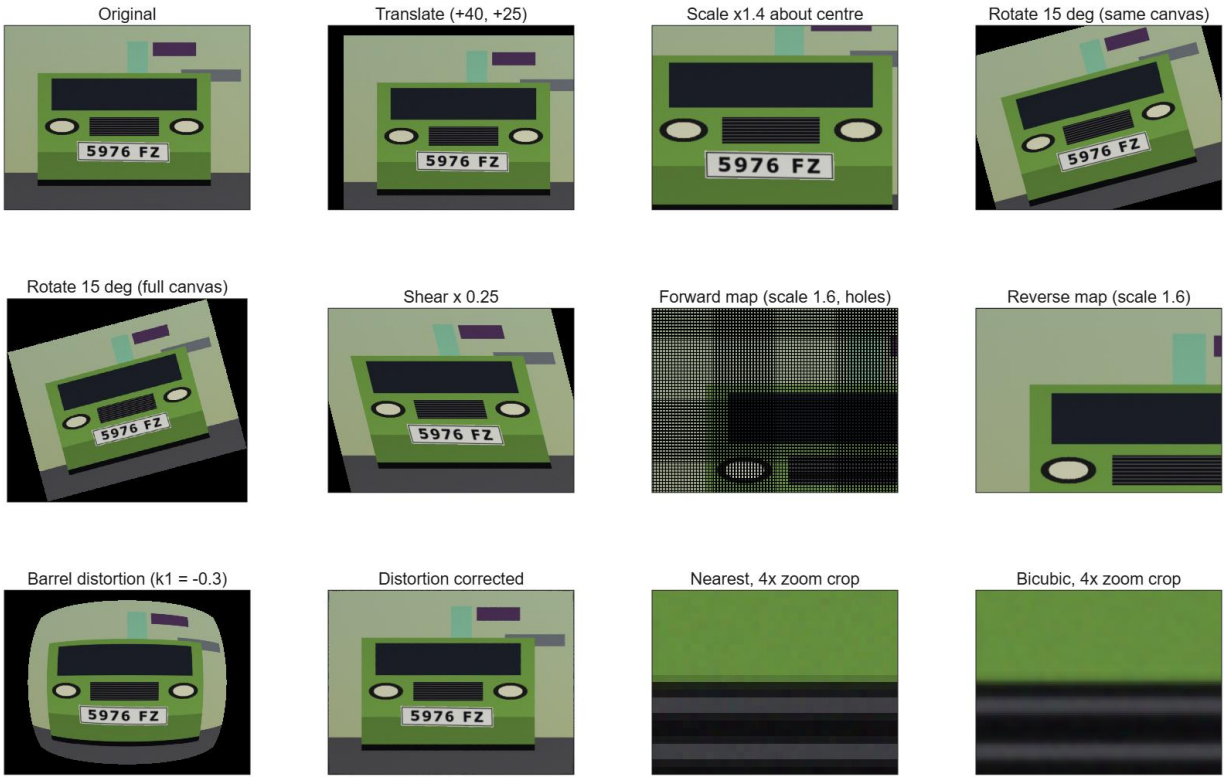


Figure 12. Geometric transformations, mapping and interpolation on the synthetic scene.

5.4.2 Interpolation

Figure 13 enlarges a 20 × 50 pixel patch of the plate text eight times with each method. Nearest-neighbour interpolation shows the original pixels as blocks, bilinear interpolation smooths them, and bicubic interpolation keeps the strokes sharper. For a quantitative comparison the scene was shrunk to half size and enlarged again, and rotated by 10° and back by −10°; Table 14 gives the PSNR against the original (central region for the rotation, to exclude the corners that leave the image) and the time for enlarging the 640 × 480 image by a factor of two.

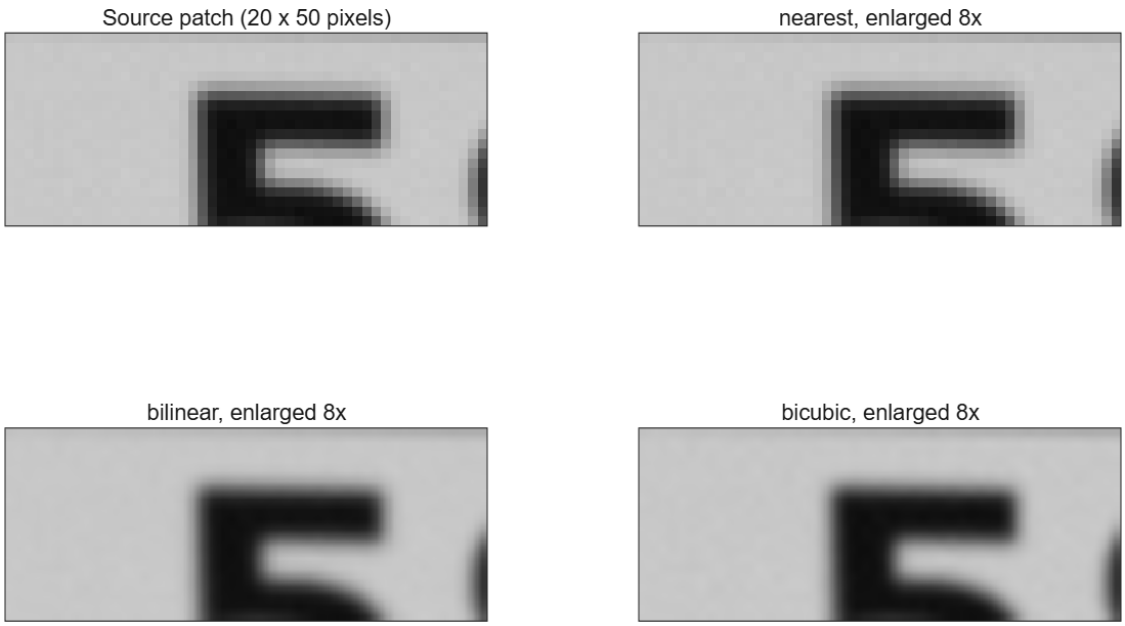


Figure 13. Interpolation methods on a patch of the licence plate text, enlarged 8 times.

Table 14. Quality and cost of the interpolation methods.

Method	Shrink and enlarge (dB)	Rotate and back (dB)	Enlarge 2× (ms)
Nearest	30.4	39.0	45
Bilinear	31.7	38.8	139
Bicubic	33.3	45.6	1722

5.4.3 Forward and reverse mapping

A white image was scaled by 1.6 with forward mapping (nearest and bilinear splatting) and with reverse mapping, and the fraction of output pixels that received no value was counted. Forward mapping with nearest-neighbour rounding left 60.8 % of the pixels empty, close to the 60.9 % that follows from the area ratio $1 - 1/1.6^2$; bilinear splatting left 0.0 % at this scale, and reverse mapping left 0.0 %. The effect can be seen in the lower panels of Figure 12.

5.4.4 Radial distortion

A grid of straight lines was distorted with barrel ($k_1 = -0.15$) and pincushion ($k_1 = 0.3$) distortion and corrected again with the same parameters (Figure 14). The lines become straight again; Table 15 gives the PSNR of the central region against the original grid before and after the correction.

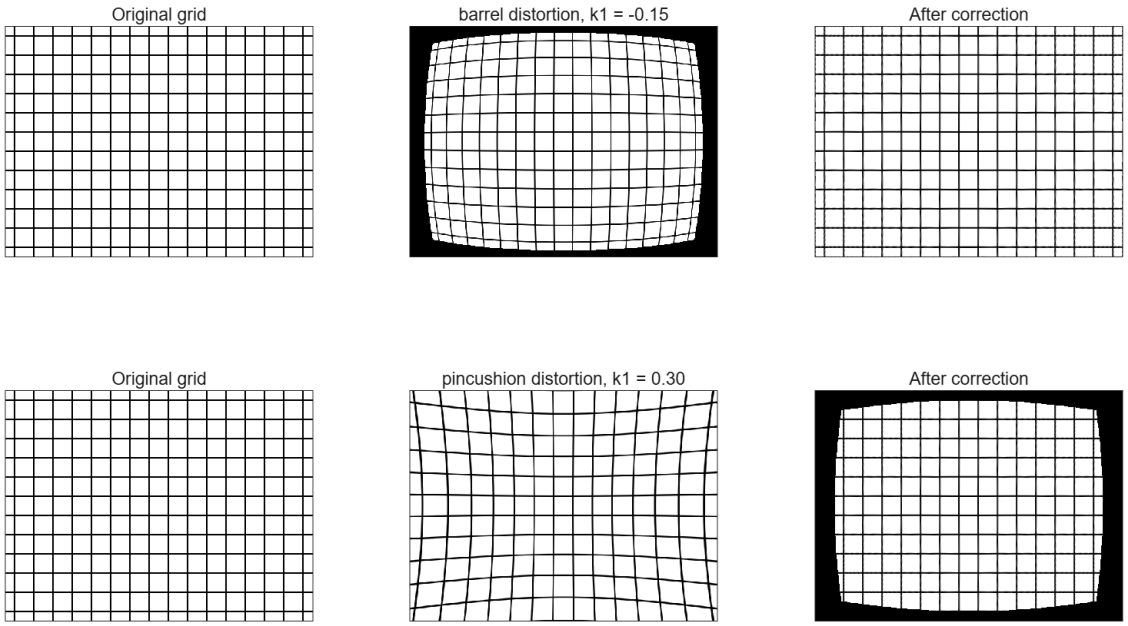


Figure 14. Barrel (top) and pincushion (bottom) distortion of a grid and its correction. Points of the distorted image that have no source point are black.

Table 15. PSNR of a thin-line grid (central region). For the synthetic scene, simulating and correcting a distortion with $k_1 = -0.25$ gave 38.5 dB.

Distortion	Distorted vs. original (dB)	Corrected vs. original (dB)
Barrel, $k_1 = -0.15$	8.8	21.7
Pincushion, $k_1 = 0.3$	8.1	22.6

5.4.5 Image registration

The scene was rotated by 8° and shifted by (12, −7) pixels and registered back to the original. Figure 15 shows the images and the differences. The feature-based method found 35 matches of which 32 were inliers, estimated the transformation with a rotation error of 0.04° and a mean error of 0.54 pixels at a grid of test

points, and improved the PSNR in the central region from 14.0 to 35.5 dB in 0.7 s. Phase correlation recovered a pure shift of (9, −5) pixels exactly (found: -9, 5; expected: -9, 5; the sign is that of the correction).

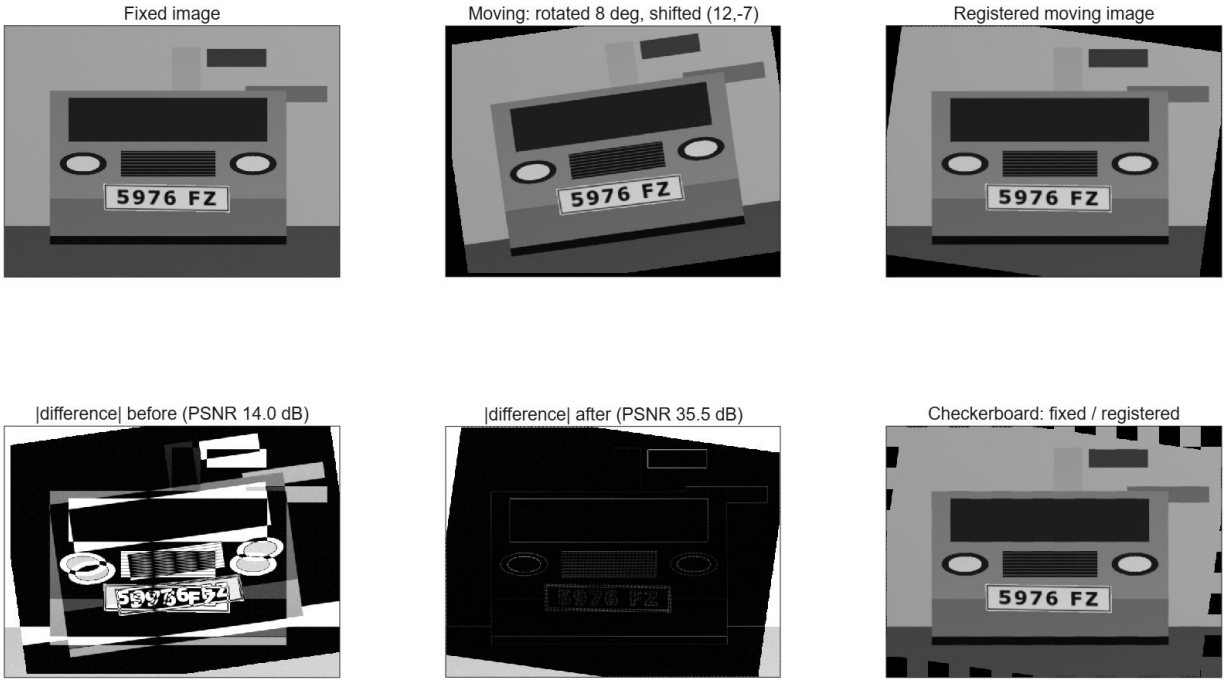


Figure 15. Image registration by feature matching.

5.5 Discussion

- **Bicubic interpolation is the most accurate but the most expensive.** It gave the best PSNR in both round trips (33.3 dB against 31.7 dB for bilinear when resizing, and 45.6 dB against 38.8 dB when rotating), and the sharpest enlarged strokes, but took 12.4 times as long as bilinear interpolation in this implementation. For nearest-neighbour the rotation round trip (39.0 dB) was as good as for bilinear (38.8 dB) because bilinear interpolation blurs a little at each of the two steps, but the blocky result of nearest-neighbour enlargement is visible in Figure 13. The recogniser uses bilinear interpolation, which is a good compromise for resizing and de-skewing characters.
- **Reverse mapping is necessary for enlargement.** Forward mapping with rounding left 60.8 % of the output empty at scale 1.6. Splatting each input pixel on its four neighbours filled all holes at this scale, but it is a more complicated and slower method, and it would leave holes again for larger scale factors (not tested).
- **Distortion correction is accurate where a solution exists.** The correction removes the curvature of the lines; the remaining PSNR difference to the original grid comes from the interpolation blur of thin lines (a thin-line grid is a severe test, because a sub-pixel shift changes many pixels). The inverse of the distortion model does not exist for the outer part of the image under strong barrel distortion (the largest reachable radius for $k_1 = -0.3$ is about 0.70 of the corner radius); the implementation detects this and leaves those pixels black. A first version used a fixed-point iteration for the inverse and produced noise in that region, which was found when preparing the figure for this report and fixed with Newton's method (Table 28).
- **Registration works for moderate motion.** The feature-based method recovered a rotation of 8° and a shift to a mean error of 0.54 pixels. It relies on corner patches that are not rotation invariant, so it will fail for large rotations (we did not test where the limit is), and it needs enough texture. Phase correlation is exact for pure translations and very fast but cannot handle rotation.
- **Limits.** All measurements use one synthetic scene and one transformation per test, so they show that the functions are correct, not how they behave on arbitrary photographs.

6 License Plate Recognition with an SVM

This chapter applies the three modules to the project topic. It describes the recogniser stage by stage, explains the support vector machine, documents how the training data were produced and presents the training experiments, the end-to-end evaluation and an analysis of the errors.

6.1 Problem statement and overview

The input is a colour or gray-scale photograph that contains one roughly frontal licence plate with dark characters on a light background. The output is the plate text, the plate position and a confidence per character. Figure 1 showed the data flow; Figure 16 shows every intermediate result for one test image, as produced by `showPipeline`.



Figure 16. The recogniser on one synthetic test scene (true plate ZWNG044, read ZWNG044, font Trebuchet MS bold, a font held out from training).

6.2 Plate detection

Detection uses only Modules 1–3 and needs no training. It exploits that the characters of a plate create a dense band of vertical strokes. `detectPlate` performs the following steps:

1. **Resize** the photo to a width of 640 pixels with bilinear interpolation (Module 3), so that all size thresholds are relative to a fixed scale.
2. **Enhance:** contrast stretch between the 1 % and 99 % percentiles and a 5 × 5 Gaussian filter with σ = 1 (Module 1).
3. **Vertical edges:** absolute horizontal Sobel response, normalised to [0, 1] and thresholded with the larger of Otsu's level and 0.06 (Module 2).
4. **Morphology:** a closing with a 3 × 45 pixel rectangle (7 % of the width) joins the strokes of neighbouring characters, also across the gap between character groups such as "ABC" and "1234"; an opening with a small rectangle removes thin clutter; holes are filled (Module 2).

5. **Candidates:** connected components whose bounding box has an aspect ratio between 2 and 13 (the blob covers only the text row, so it is wider than the plate), a height of at least 1.2 % of the image height, a width of at least 7 % of the width and an extent of at least 0.45.
6. **Score and crop:** each candidate is scored by the product of its edge density, the Otsu separability η of its gray values (text on a plate is bimodal), a preference for an aspect ratio near 6 and a colour factor 0.3 + (fraction of white or yellow pixels). The best candidate is cropped with a margin of 6 % of its width and 35 % of its height, which covers the plate border.

The width of the closing was the only parameter that needed tuning. Table 16 shows the detection rate for several widths on 60 synthetic scenes of difficulty 0.6 with unseen fonts: a closing that is too narrow cannot bridge the gap between character groups and the detector boxes only half of the plate, a closing that is too wide merges the plate with neighbouring structures.

Table 16. Effect of the closing width on plate detection (development experiment).

Closing width (fraction of image width)	Plate found (IoU > 0.5)	Plate fully inside the box	No candidate	Mean IoU
0.04	44 / 60	42 / 60	0	0.68
0.06	59 / 60	58 / 60	0	0.78
0.07 (chosen)	**59 / 60**	**60 / 60**	**0**	**0.78**
0.09	58 / 60	60 / 60	0	0.76
0.12	54 / 60	60 / 60	0	0.72

6.3 Character segmentation

`segmentChars` turns the plate crop into a list of normalised character images:

1. The plate is resized to a height of 100 pixels and smoothed with a 3 × 3 Gaussian filter.
2. It is binarised with Otsu's threshold. Both polarities (dark characters, bright characters) are tried and the one that produces the larger group of characters is kept.
3. The foreground is labelled with 8-connectivity (`labelComponents`) and components are discarded when they touch the image border, are lower than 25 % or higher than 95 % of the plate height, or are wider than 1.5 times their height.
4. **Grouping:** the characters of a plate have similar heights and lie on one line. For every remaining component, the components whose height is between 0.65 and 1.6 times its height are collected, a line is fitted through their centroids (every pair of components proposes a line; a component is an inlier when it lies within 0.30 character heights of the line and the line is flatter than 25°), and the largest group wins.
5. **De-skew:** if the fitted line is tilted by 1° or more, the plate is rotated by the opposite angle (Module 3, reverse mapping with bilinear interpolation, border colour as fill) and steps 2–4 are repeated.
6. **Normalise:** every character is cut out with only its own connected component as mask, cropped to its bounding box, scaled with its aspect ratio preserved so that its longer side is 80 % of a 32 × 32 canvas, and centred. The same function (`normalizeChar`) is used for training characters, so training and recognition see identical geometry.

Two alternatives were tested on 150 scenes (difficulty 0.5, unseen fonts, SVM trained on other fonts). Adding a local adaptive threshold as a second binarisation candidate was expected to help against uneven lighting,

but it lowered the accuracy, so the default is Otsu only; widening the accepted height range (for characters such as Q with a long tail) made no difference on this data and was kept because it is harmless (Table 17).

Table 17. Segmentation variants compared on the same scenes (development experiment).

Configuration	Correct character count	Exact plate (no layout)	Exact plate (layout known)
Otsu only, height range 0.75-1.33	94.7 %	79.3 %	87.3 %
Otsu only, height range 0.65-1.6 (chosen)	**94.7 %**	**79.3 %**	**87.3 %**
Otsu + adaptive, height range 0.75-1.33	92.7 %	72.7 %	83.3 %
Otsu + adaptive, height range 0.65-1.6	91.3 %	71.3 %	82.0 %
Adaptive only, height range 0.75-1.33	92.0 %	72.0 %	84.0 %

6.4 Feature extraction: HOG

A raw 32 × 32 image has 1024 values that change a lot with the font, whereas the arrangement of stroke directions is much more stable. The histogram of oriented gradients (Dalal and Triggs) describes exactly this. `hogFeatures` computes:

1. central-difference gradients G_x, G_y (kernel [−1 0 1]) and the magnitude and the unsigned orientation (0–180°) of every pixel;
2. for every 8 × 8 pixel cell (4 × 4 cells), a histogram of 9 orientation bins in which each pixel votes with its gradient magnitude, split linearly between the two nearest bins;
3. overlapping blocks of 2 × 2 cells (stride one cell, 3 × 3 blocks) whose 36 values are normalised with the L2-Hys scheme: $v \leftarrow v/\sqrt{\|v\|^2 + \varepsilon}$, clip at 0.2, normalise again, which makes the descriptor robust to contrast changes.

The descriptor has 3 × 3 × 36 = 324 values per character.

6.5 Support vector machine classifier

6.5.1 Binary soft-margin SVM

Given training vectors x_i with labels $y_i \in \{+1, -1\}$, a support vector machine looks for the separating hyperplane with the largest margin. Allowing a few violations ξ_i of the margin (soft margin) leads to the primal problem

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \qquad s.t.\ y_i\left(w^{\mathrm{T}}\varphi(x_i) + b\right) \geq 1 - \xi_i,\ \xi_i \geq 0 \tag{23}$$

where the constant C trades margin width against violations. Through its Lagrangian dual the problem only needs inner products of the mapped points, which are replaced by a kernel function $K(x_i, x_j) = \phi(x_i)\cdot\phi(x_j)$ (the "kernel trick"):

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j K(x_i, x_j) \qquad s.t.\ 0 \leq \alpha_i \leq C,\ \sum_i \alpha_i y_i = 0 \tag{24}$$

The training points with $\alpha_i > 0$ are the *support vectors*; the classifier is $f(x) = \Sigma\, \alpha_i\, y_i\, K(x_i, x) + b$ and the predicted class is the sign of f. We use the radial basis function (RBF) kernel, which measures similarity by distance and lets the boundary curve around classes (Figure 17 below shows the effect on a toy problem):

$$K(x, x') = e^{-\gamma\|x - x'\|^2} \tag{25}$$

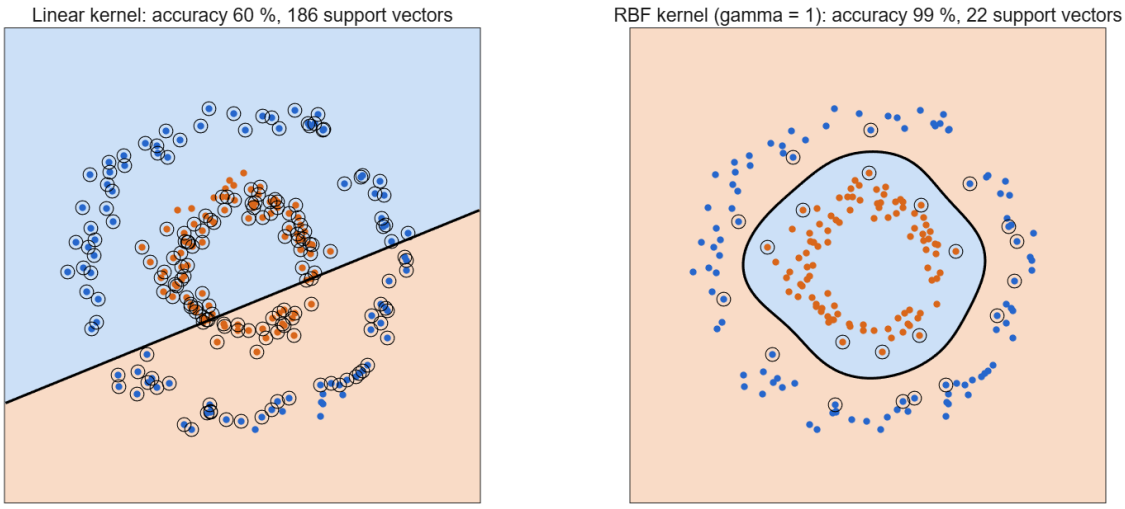


Figure 17. A linear and an RBF kernel on two concentric rings. The linear SVM reaches 60 % on held-out points, the RBF SVM 99 %. Circled points are support vectors.

6.5.2 Training with sequential minimal optimisation (SMO)

The dual problem is a quadratic programme with one equality constraint. SMO (Platt, 1998) solves it by repeatedly optimising two multipliers α_i and α_j analytically while all others stay fixed, which keeps the constraint $\Sigma\, \alpha_i\, y_i = 0$ satisfied. With the prediction errors $E = f(x) - y$ the update of the second multiplier is

$$\alpha_j^{new} = \alpha_j + \frac{y_j(E_i - E_j)}{\eta}, \quad \eta = K_{ii} + K_{jj} - 2K_{ij} \tag{26}$$

The result is clipped to the interval [L, H] that the box constraint $0 \le \alpha \le C$ and the equality constraint allow, and α_i is adjusted by $y_i\, y_j\, (\alpha_j - \alpha_j^{new})$. The pair is chosen with the maximal-violating-pair rule and the second-order criterion of Fan, Chen and Lin: i is the point that violates the optimality (KKT) conditions most, and j is the partner that promises the largest decrease of the objective. Training stops when the largest violation is below 10^{-3}. The bias is the mean of −(error) over the free support vectors ($0 < \alpha < C$). `svmTrainBinary` keeps the full kernel matrix of the (small) pair problem in memory and updates the error vector with two columns of it per iteration.

6.5.3 Multi-class classification

The 36 characters are separated by the *one-vs-one* scheme: one binary SVM is trained for each pair of classes, $K(K-1)/2 = 630$ machines, each on the examples of its two classes only. This keeps every optimisation problem small (here about 500 samples) and is usually faster than one-vs-rest, whose problems contain all samples. A new character is classified by letting every machine vote for one of its two classes; the class with the most votes wins and ties are broken by the sum of the decision values. The share of the 35 duels that the winner won is reported as a confidence. The support vectors of all machines are stored once and shared (`svmTrain`, `svmPredict`), so a prediction needs one kernel matrix between the test vectors and the shared support vectors.

Table 18. Functions of the SVM (toolbox/svm) and of the licence plate recogniser (toolbox/lpr).

Function	Purpose
`f = hogFeatures(I, cellSize, nBins)`	Histogram of Oriented Gradients descriptor (Dalal & Triggs).
`K = kernelMatrix(A, B, kp)`	Kernel (Gram) matrix between the rows of A and the rows of B.
`[pred, conf, votes, score] = svmPredict(model, X)`	Classify rows of X with a model from SVMTRAIN (one-vs-one voting).
`model = svmTrain(X, labels, varargin)`	Multi-class SVM (one-vs-one) built from binary SMO classifiers.
`m = svmTrainBinary(X, y, C, kp, tol, maxIter)`	Soft-margin binary SVM trained with Sequential Minimal Optimisation.

Function	Purpose
`N = augmentGlyph(g)`	Turn a clean font glyph into a realistic, normalised training sample.
`tpl = buildGlyphTemplates(cacheFile)`	Render the characters 0-9 and A-Z in many fonts (training material).
`[bbox, plateImg, dbg] = detectPlate(img, varargin)`	Locate the license plate in a photo using Modules 1-3 (no learning).
`R = evaluateLPR(model, varargin)`	Measure the recognition pipeline on freshly generated synthetic scenes.
`[X, y, imgs] = generateCharDataset(tpl, variantIdx, nPerClass, seed)`	Synthetic character dataset (HOG features + labels).
`[img, gt] = makeScene(tpl, variantIdx, varargin)`	Synthetic street photo of a car with a license plate (test data generator).
`N = normalizeChar(ink, outSize, fill)`	Crop a binary character to its bounding box and fit it into a square canvas.
`[txt, fmt] = randomPlateText(fmt)`	Random plate number following a letter/digit format.
`res = recognizePlate(img, model, varargin)`	Full pipeline: photo -> plate location -> characters -> SVM -> text.
`[plate, info] = renderPlate(txt, tpl, v, varargin)`	Draw a license plate image for a given text using font glyph templates.
`[chars, boxes, dbg] = segmentChars(plateImg, varargin)`	Cut a plate image into normalised character images.
`fig = showPipeline(img, res, visible)`	Figure with every stage of the plate recognition pipeline.
`model = trainLPRModel(varargin)`	Train the character-recognition SVM and save it.

6.6 Training data

A photograph collection with labelled plates was not available, so the training characters are generated. This is a deliberate design decision with an important consequence: the classifier is trained on *synthetic* characters and its accuracy on real photographs is unknown.

6.6.1 Glyph rendering

`buildGlyphTemplates` draws the 36 characters in every font found on the computer (21 font variants, regular and bold) into a hidden MATLAB figure as a 6 × 6 mosaic, captures the pixels with `getframe` and cuts each glyph out (Figure 18). The result is cached because rendering takes about 15 seconds.

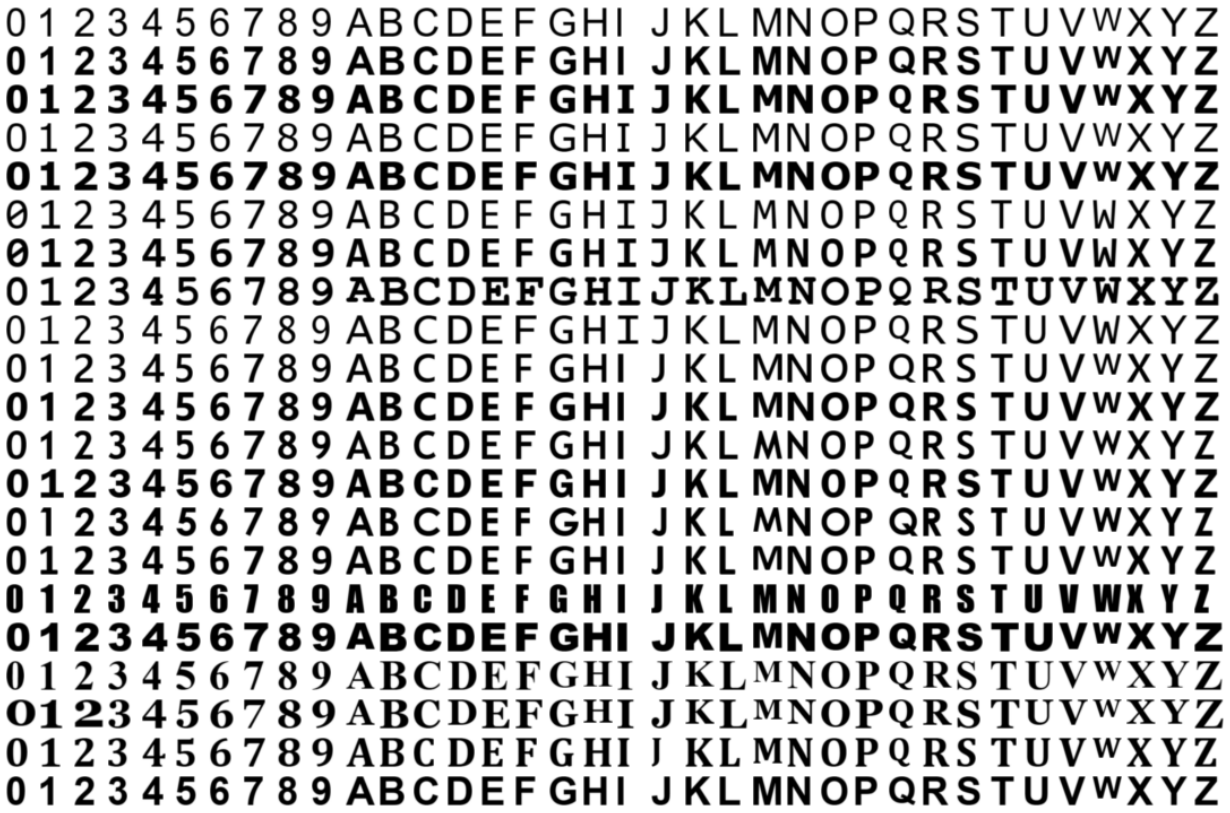


Figure 18. The rendered glyphs: one row per font variant, 36 characters per row.

6.6.2 Degradation

`augmentGlyph` turns a clean glyph into a training sample that resembles a character cut from a photographed plate: random height of 20–60 pixels, random shear (±0.18) and rotation (±8°) with Module 3, random ink and paper brightness, an optional down- and up-scaling (low resolution), Gaussian blur (σ = 0.3–1.4 pixels) and noise (up to 0.07), followed by Otsu binarisation, removal of specks and the same normalisation that the recogniser uses. Figure 19 shows examples; each column is one class and each row a different random degradation.



Figure 19. Training samples after degradation and normalisation (3 per class; 0–9 then A–Z).

6.6.3 Training and test sets

The fonts are split: 17 fonts are used for training and 4 fonts (Verdana Bold, Calibri Bold, Trebuchet MS Bold and Courier New Bold) are held out and never seen by the SVM. This tests whether the classifier learned the shapes of the characters rather than the shapes of particular fonts. The training set has 9,000 images (250 per class). Two test sets are generated with other random seeds: 1,440 images in the unseen fonts and 720 new images in the training fonts.

6.6.4 Synthetic test scenes

The end-to-end tests use whole scenes created by `makeScene`: a gradient background with rectangles as buildings, a road, a car with windshield, headlights, grille and bumper, and a plate that is rendered by `renderPlate` (white or yellow, with a border and a random font and spacing). The plate is scaled to 150–260 pixels width, warped into the scene by rotation and shear with Module 3, and the whole image receives uneven lighting, blur and noise. A difficulty parameter $h \in [0, 1]$ scales these disturbances:

Table 19. Disturbances of the synthetic scenes as a function of the difficulty h.

Disturbance	Range	h = 0.2	h = 0.5	h = 0.8
Plate rotation	uniform in ±(2 + 8h) degrees	±3.6°	±6.0°	±8.4°
Plate shear	uniform in ±0.1 h	±0.02	±0.05	±0.08
Brightness gain	1 − 0.45 h · u, u uniform in [0, 1]	≥ 0.91	≥ 0.78	≥ 0.64
Camera blur (σ, pixels)	0.3 + h · u	≤ 0.5	≤ 0.8	≤ 1.1
Noise (σ)	0.005 + 0.03 h · u	≤ 0.011	≤ 0.020	≤ 0.029

6.7 Training and model selection

Figure 20 (left) and Table 20 show how the accuracy depends on the number of training images per class. The accuracy on unseen fonts rises from 92.0 % with 10 images per class to 97.2 % with 250, with diminishing returns beyond 50. Fonts seen in training score higher (98.6 % at 250 images), the difference of about one percentage point is the price of the unseen fonts.

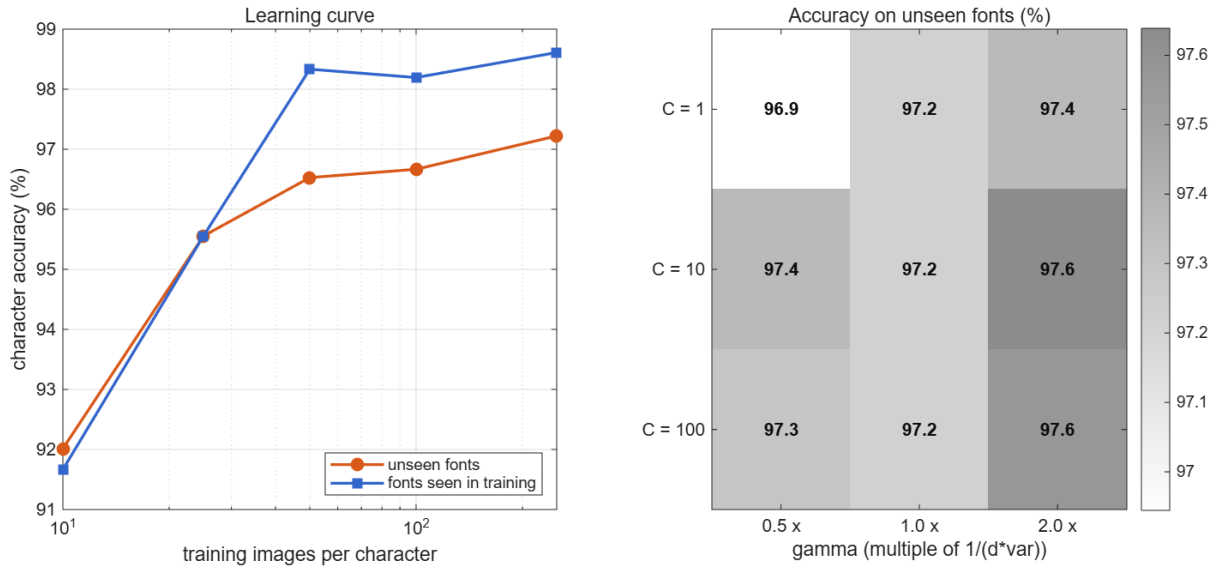


Figure 20. Left: learning curve of the RBF-SVM. Right: accuracy on the unseen fonts (%) for a grid of C and γ.

Table 20. Learning curve (C = 10, RBF kernel with γ = 1/(d · var(X))).

Images per class	Accuracy, unseen fonts	Accuracy, seen fonts	Support vectors	Training time (s)
10	92.0 %	91.7 %	353	0.8
25	95.6 %	95.6 %	781	1.4
50	96.5 %	98.3 %	1299	1.7
100	96.7 %	98.2 %	2017	2.5
250	97.2 %	98.6 %	3587	7.9

Three kernels were compared on the full training set (Table 21). The differences are small. With 1,440 test images the standard error of an accuracy near 97 % is about 0.4 percentage points, so the 0.5-point advantage of the RBF kernel over the linear kernel is within the sampling error. The HOG feature space is already almost linearly separable for this task. We keep the RBF kernel as the default because it gave the highest accuracy and is the standard choice, but a linear SVM would be nearly as accurate with fewer support vectors (2749 against 3587), a point to remember when speed matters.

Table 21. Kernel comparison (C = 10, 250 images per class).

Kernel	Unseen fonts	Seen fonts	Support vectors	Training time (s)
Linear	96.7 %	97.8 %	2749	6.7
RBF (default)	97.2 %	98.6 %	3587	8.6
Polynomial, degree 3	97.2 %	97.9 %	3129	10.7

The grid over $C \in \{1, 10, 100\}$ and $\gamma \in \{0.5, 1, 2\}$ times the default (right panel of Figure 20) is nearly flat: all nine combinations lie between 96.9 % and 97.6 %. The classifier is therefore not sensitive to these hyper-parameters, and the defaults $C = 10$ and $\gamma = 1/(d \cdot \mathrm{var}(X)) = 0.19$ were kept. The final model has 630 binary machines that share 3,587 support vectors and trains in a few seconds.

6.8 End-to-end evaluation

The complete recogniser was evaluated with `evaluateLPR` on 100 new synthetic scenes for each of three difficulty levels. The scenes use the four held-out fonts and the SVM was trained without them, so neither the fonts nor the scenes were seen during development of the classifier. Each scene is read twice: once with the plain 36-class decision and once with the letter/digit layout of the plate given (for example “LLL DDDD”), in which case the SVM decision of a character is restricted to digits or to letters. Table 22 gives the results.

Table 22. End-to-end results on 100 synthetic scenes per difficulty level (unseen fonts). Character accuracy is 1 − (edit distance / plate length), averaged.

Measure	h = 0.2	h = 0.5	h = 0.8
Plate located (IoU > 0.5)	98.0 %	99.0 %	100.0 %
Correct number of characters	96.0 %	93.0 %	89.0 %
Character accuracy, no layout	96.5 %	94.5 %	92.1 %
Character accuracy, layout known	97.7 %	95.8 %	94.2 %
Exact plate, no layout	**79.0 %**	**75.0 %**	**67.0 %**
Exact plate, layout known	**87.0 %**	**83.0 %**	**79.0 %**
Time per image	0.36 s	0.44 s	0.41 s

Detection is almost perfect at every level. The accuracy falls steadily with difficulty because the segmentation fails more often (96 % to 89 % correct character counts) and the characters become harder to classify. Knowing the layout improves the exact-plate rate by 8 to 12 percentage points. Figure 21 shows the first twelve scenes of the medium level.



Figure 21. First twelve scenes of the medium difficulty level. Titles show the true plate and the SVM reading (green: correct, red: wrong); the red box is the detected plate.

Speed. Table 23 breaks the time per image down by stage (mean of 150 scenes). Detection dominates; extracting the HOG features and classifying all characters of a plate together take only about 14 ms.

Table 23. Computation time per stage.

Stage	Mean time (ms)
Plate detection (640 × 480 image)	323
Character segmentation	25
HOG and SVM classification	14
Total	**362**

6.9 Error analysis

6.9.1 Single-character errors

Figure 22 is the confusion matrix of the SVM on the 1,440 test characters of the unseen fonts (overall accuracy 97.2 %). Almost all mass lies on the diagonal. The most frequent confusions are E→B (6 of 40), F→E (6 of 40), 0→O (5 of 40), T→W (4 of 40), P→E (3 of 40). They are pairs of characters that share their main strokes (E/B/F, T/W, 0/O); we did not analyse the causes further.

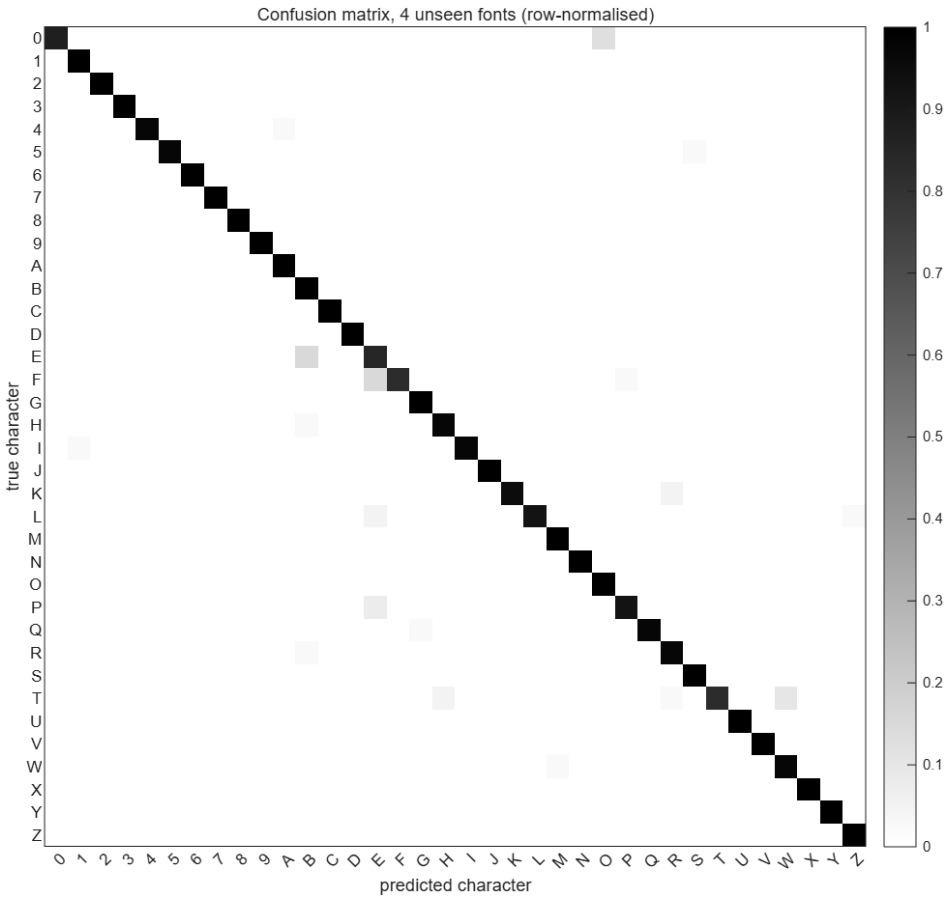


Figure 22. Confusion matrix of the SVM on four unseen fonts (rows normalised).

6.9.2 Plate-level errors

To see why whole plates fail, 150 further scenes of medium difficulty were read without a layout hint and every failure was assigned to one cause (Table 24). A failure is a *look-alike confusion* if all wrong characters belong to a pair that is visually almost identical (O/0, I/1, B/8, S/5, Z/2, D/0, Q/0, G/6, Q/O, D/O, U/V, I/L, G/0, C/G).

Table 24. Outcome of 150 scenes at difficulty 0.5, no layout hint.

Outcome	Scenes	Share
Plate read exactly	123	82.0 %
Look-alike confusion only (O/0, I/1, B/8, …)	10	6.7 %
Other character substitution	11	7.3 %
Wrong number of characters (segmentation)	5	3.3 %
Box found but not on the plate	1	0.7 %
No plate candidate	0	0.0 %

Of the 27 failures, 10 (37 %) are look-alike confusions, which the layout hint removes in the cases where the confused characters belong to different types (letter versus digit); 5 (19 %) are segmentation errors. Figure 23 shows plate crops of failed cases. The crops shown there are character substitutions. The segmentation errors that were examined during development (not shown) were characters that merged with the plate frame after strong rotation and characters with a long tail (Q) that were lost.



Figure 23. Plate crops of failed scenes (true text, SVM reading, failure type).

Table 25. Exact-plate rate per font in the error-analysis scenes. The counts per font are small (about 40), so differences of ten points are not significant.

Font (held out from training)	Scenes	Exact plate, no layout
Calibri bold	29	96.6 %
Courier New bold	39	64.1 %
Trebuchet MS bold	46	95.7 %
Verdana bold	36	72.2 %

What the evaluation does not show

All scenes are synthetic and were generated by code that the authors wrote, so they share assumptions with the recogniser (a roughly frontal plate, a clean rectangular frame, no glare, no dirt, no motion blur, no perspective foreshortening). Real photographs will contain disturbances that are not modelled, and the accuracy on them is unknown. The numbers above are a regression test and a way to find weaknesses, not an estimate of field performance.

7 Graphical User Interface

7.1 Overview

The assignment asks for a MATLAB interface in which a user uploads an image, chooses a processing technique and sees the result. `LPR_Toolbox` (started with the command `LPR_Toolbox`) provides this with 48 operations in four tabs (Table 26). The window shows the original image and the result side by side and a third plot below them that depends on the tab: the gray-level histograms of input and result for Module 1, and the normalised character images for the plate recogniser.

Table 26. The four tabs of the interface.

Tab	Operations	Examples
1 Enhance	17	histogram equalisation and matching, brightness/contrast, gamma, log, stretch, mean/Gaussian/median filters, Laplacian/high-pass/unsharp sharpening, FFT low/high/band-pass, noise generators
2 Segment	16	Sobel/Prewitt/Canny, Otsu and adaptive threshold, region growing, colour segmentation (white, yellow, blue, red), dilation, erosion, opening, closing, gradient, shape-based segmentation
3 Geom.	15	translation, scaling, rotation (same/full canvas), shear, affine, forward and reverse mapping, nearest/bilinear/bicubic resize, radial distortion (simulate/correct), image registration (features/phase correlation)
4 Plate	–	plate detection, character segmentation and SVM recognition with optional layout; display of every pipeline stage; training of a new SVM

A typical session:

1. Press **Open image…**, or **Synthetic car photo** to generate a test scene. Very large photos are scaled down to at most 1000 pixels so that all operations stay responsive.
2. Choose a tab and an operation in the drop-down list. The sliders below it change to the parameters of that operation (for example sigma of a Gaussian filter, the angle of a rotation or the tolerance of region growing) and show their current value.
3. Press **Apply**. The result and an information line (operation, run time and a short description) appear; for Module 1 the histograms are drawn as well.
4. Press **Use result as input** to chain operations (for example histogram equalisation followed by Canny edges), **Reset to original** to go back, **Open reference…** to load the second image needed by histogram matching and registration, and **Save result…** to write the result to a file.
5. In tab 4, press **Recognise plate**. The detected plate is outlined on the input, the segmented plate with the character boxes and the 32 × 32 character images are shown, and the text appears in large letters. **Show pipeline stages** opens the figure of Chapter 6 for the current image.

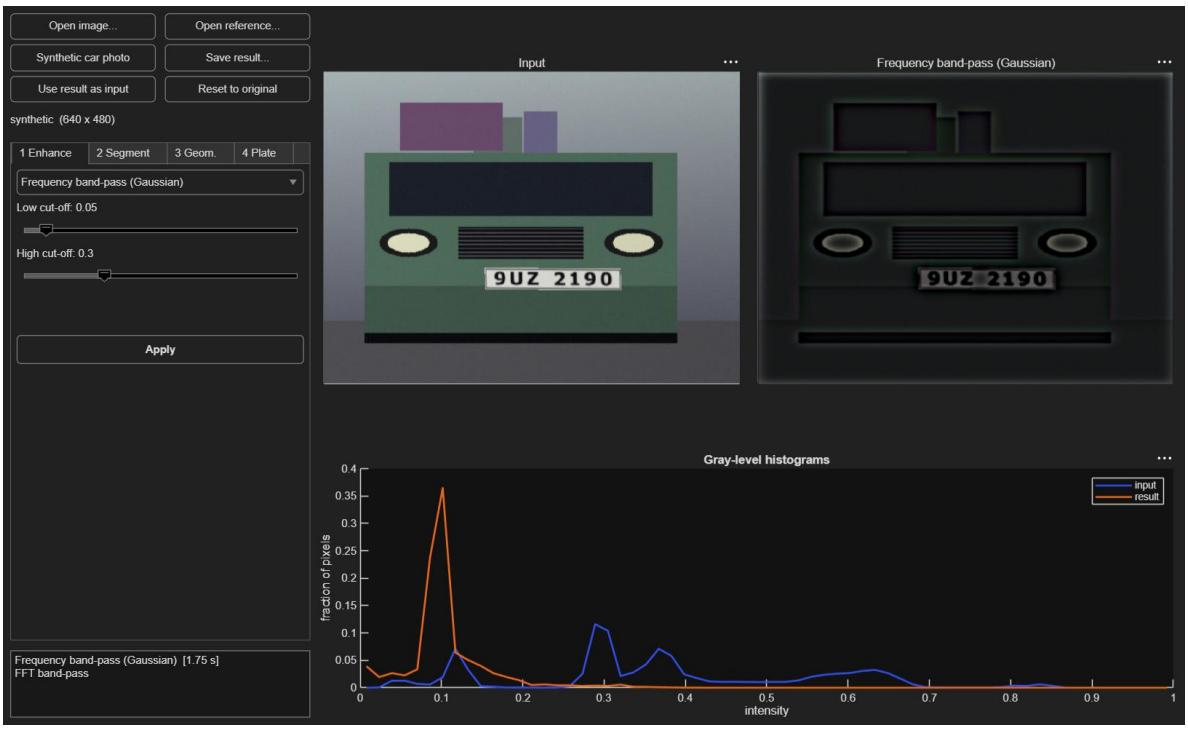


Figure 24. Tab 1 (enhancement): a Gaussian band-pass filter on a test scene. The lower plot shows the gray-level histograms of input and result.

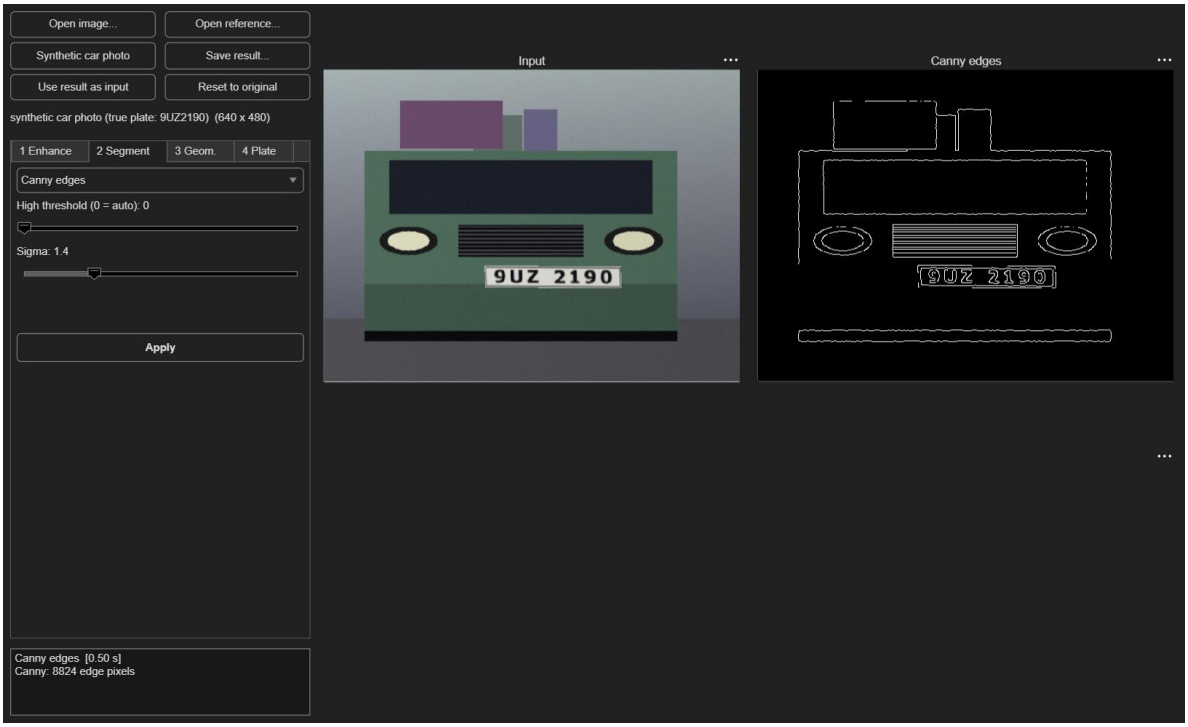


Figure 25. Tab 2 (segmentation): Canny edge detection with its two sliders.

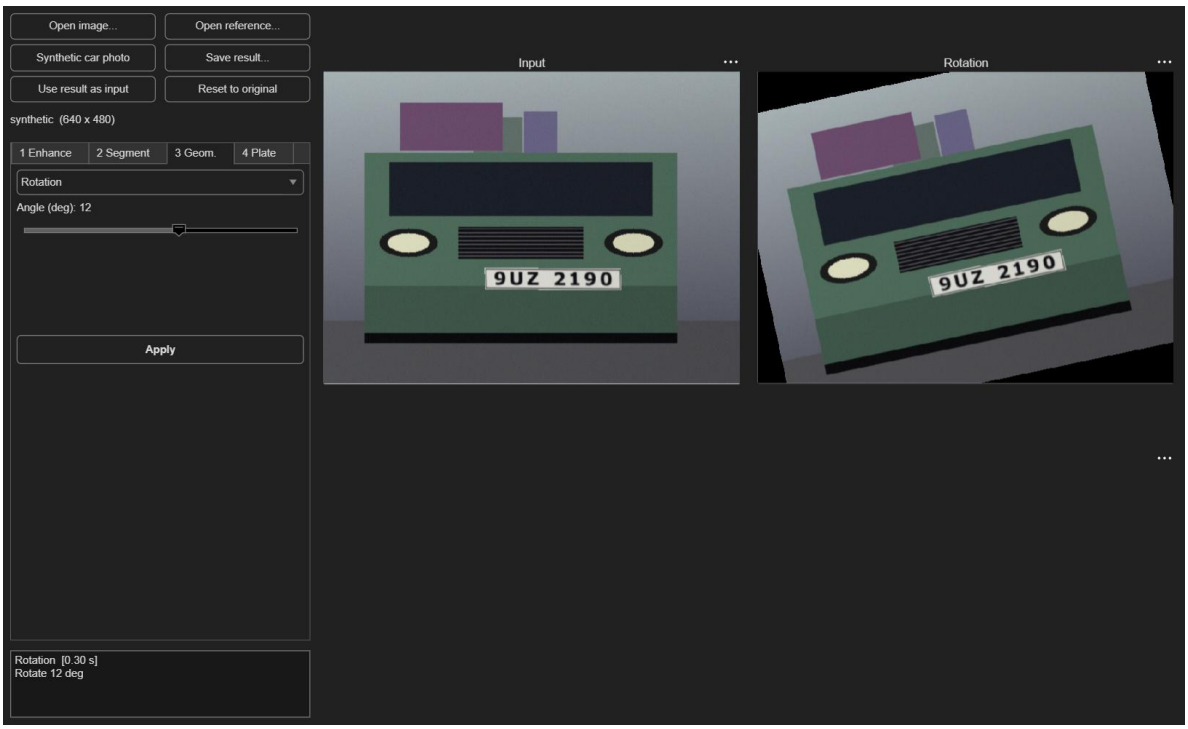


Figure 26. Tab 3 (geometry): rotation by 12°.

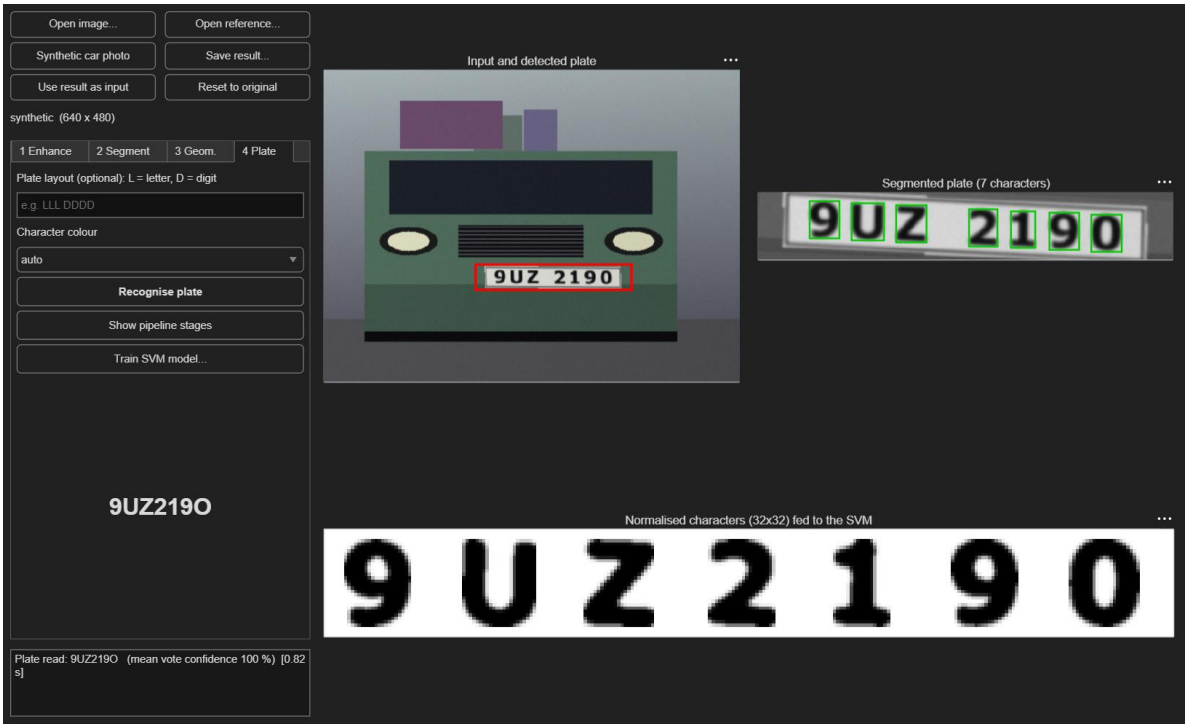


Figure 27. Tab 4 (plate recognition): detected plate, segmented characters and the 32 × 32 images that are passed to the SVM. The window uses the dark colour theme of the operating system.

7.2 Implementation

The interface is built programmatically with `uifigure`, `uigridlayout`, `uitabgroup`, `uislider` and `uiaxes`. All operations are described in one table, `toolboxOperations`, in which every entry has a module number, a name, up to three parameters (label, range, default) and a function handle that maps an input image and the parameter values to a result image and an information string. The interface creates its drop-downs and sliders from this table, and the test suite runs every entry, so adding a technique to the toolbox means adding one entry. Errors in an operation are caught and shown in a dialog instead of stopping the interface.

For automated testing the interface returns a structure of handles (`app = LPR_Toolbox('Visible','off')`) with the functions `loadImage`, `applyByName` and `recognize`. `applyByName` sets the tab, drop-down and sliders exactly as a user would and then runs the operation. This was used to run all operations once on a test scene (all of them completed without error) and to take the screenshots in this chapter.

8 Testing and Validation

8.1 Strategy

Because no toolbox function can serve as a reference, the correctness of every module is checked by `run_tests`, a script of 39 automatic checks. They use four kinds of evidence:

- **Reference calculations:** connected-component labelling is compared with an independent flood fill on random images; dilation with a direct convolution; rotation by 90° with MATLAB's `rot90`; translation with array indexing; bilinear and bicubic interpolation of a linear ramp with the exact value.
- **Mathematical properties:** erosion is the dual of dilation, opening never adds and closing never removes foreground pixels, Otsu's threshold lies between the two classes of a bimodal histogram, the SMO solution satisfies $\Sigma\, \alpha_i\, y_i = 0$, a Gaussian filter leaves a constant image unchanged.
- **Behavioural checks:** histogram equalisation increases contrast, the median filter beats the mean filter on impulse noise, a low-pass filter smooths and a high-pass filter does not, forward mapping leaves holes while reverse mapping does not, the registration functions reduce the alignment error or recover a known shift, the RBF-SVM separates two concentric rings.
- **End-to-end regression:** the plate detector must find the plate of a test scene, and on 40 scenes of unseen fonts the recogniser must reach at least 90 % detection, 90 % character accuracy and 70 % exact plates with a known layout. These thresholds are deliberately below the measured values so that the test catches regressions without being fragile.

8.2 Checks

Table 27. The 39 automatic checks by group (full list in Appendix B).

Group	Checks	Examples
Common helpers	3	toDouble: uint8 -> [0,1]; toDouble: double with small overshoot is left alone; …
Module 1	8	histEqualize increases contrast; histMatch moves mean towards reference; …
Module 2	9	labelComponents matches flood-fill reference; Sobel/Prewitt/Canny find the step edge; …
Module 3	11	rotate 90 deg equals rot90; translation moves pixels; …
SVM	3	binary RBF-SVM separates concentric circles; SMO keeps sum(alpha*y) = 0; …
Licence plate pipeline	5	makeScene returns a plate box inside the image; detectPlate finds the plate (IoU > 0.5); …

At the time of writing all 39 checks pass in MATLAB 26.1.0.3312084. In addition, every one of the 48 operations of the interface was executed once on a test scene through the scripted interface without error.

8.3 Defects found and fixed

Testing found real defects during development; they are listed because they show what the checks are for and which kinds of error to expect in such code.

Table 28. Defects found by testing.

Defect	How it was found	Fix
A rotation by exactly 90° produced a canvas one pixel too large, because cos(90°) is $6 \cdot 10^{-17}$ and not 0.	Comparison with `rot90` (size mismatch).	Subtract a tolerance before the ceiling operation; add a tolerance of 10^{-6} pixel to the inside-image test of the interpolation.
The Harris corner detector returned no corners for a rotated image: the black wedges left by the rotation have very strong corners, so the threshold of 1 % of the maximum rejected every real corner.	Feature registration did not improve the alignment in the registration check.	Take the strongest local maxima instead of thresholding relative to the global maximum.
Bilinear forward mapping added a matrix to a transposed vector and crashed.	The path was only exercised when the report experiments called it; GUI and tests had only used nearest-neighbour forward mapping.	Use column accumulators; new check for bilinear forward mapping with the identity and for the holes of forward mapping.
A double image whose maximum was slightly above 1 after adding noise was divided by 255 as if it were an 8-bit image, which wrecked thresholds.	Unexpected thresholds in the training-data generator.	Rescale only when the maximum exceeds 1.5; clamp explicitly after adding noise.

9 Discussion

9.1 Summary of findings

- **Module 1.** The right enhancement depends on the degradation: equalisation raised the contrast of a flat image by a factor of 4.6 but also amplified noise in flat regions; the median filter removed impulse noise (15.3 to 37.2 dB) where linear filters could not; Gaussian filters were best against Gaussian noise; the ideal frequency filter rings and the Gaussian one does not.
- **Module 2.** On shapes with known edges, Canny was the only detector that kept an F1 score of 1.00 at noise level 0.08, while Sobel and Prewitt fell to 0.91 and 0.95; run-based labelling made connected components fast enough for interactive use; morphology cleaned the binarised plate text.
- **Module 3.** Reverse mapping with interpolation is the right way to warp images; bicubic interpolation preserved the most detail at about the cost reported in Chapter 5; radial distortion and image registration were recovered accurately on test images.
- **Recogniser.** The SVM on HOG features reads characters of fonts it never saw with about 97 % accuracy. On synthetic scenes of medium difficulty the plate is located in 99 % of the images and read exactly in 75 % (83 % with a known layout).

9.2 Limitations

The following limitations affect how far the results can be trusted and should be read together with the numbers in this report.

1. **Synthetic data only.** Both the training characters and all test scenes are generated. The recogniser has not seen a real photograph, so its real-world accuracy is unknown and probably lower. The evaluation shows that the pipeline works and where it breaks; it is not a field benchmark.
2. **Statistical uncertainty.** With 100–150 scenes per measurement the standard error of an exact-plate rate near 80 % is about 3–4 percentage points. Two independent samples of medium difficulty gave 75 % and 82 % without a layout hint, so differences of a few points between settings should not be interpreted.
3. **Dependence on the font.** The exact-plate rate varied between the four held-out fonts from about 64 % to 97 % (Table 25 in Section 6.9). Characters of a font that is very different from the 17 training fonts are read considerably worse. Which fonts are available on the computer changes the model.
4. **Look-alike characters.** Letters and digits such as O/0, I/1 and B/8 are visually identical in many fonts. Without knowing the layout of the plate these errors are unavoidable for any classifier that sees one character at a time; the layout hint is the practical remedy.
5. **Simplifying assumptions in the pipeline.** One roughly frontal plate per image, dark characters on a light plate (a bright-on-dark option exists), characters that do not touch each other or the plate frame, rotation of at most about 10° and no perspective distortion. Plates photographed at an angle need a perspective correction that is not part of the toolbox.
6. **Parameters tuned on synthetic scenes.** The closing width of the detector was selected on synthetic scenes (Section 6.2) and may need adjustment for real photographs.
7. **Speed.** One image takes 0.3–0.45 s in interpreted MATLAB, mostly in the detector. This is adequate for single images but not for real-time video.
8. **Alphabet.** Only 0–9 and A–Z are recognised.

9.3 Applications of the toolbox in real-world scenarios

The toolbox is not limited to licence plates. Each module contains techniques that are standard building blocks in practical systems; the table lists where they apply. These are applications of the *techniques*; none of them was tested in this project except licence plate recognition.

Table 29. Where the techniques of the toolbox are used in practice.

Application	Techniques from the toolbox	Role
Traffic control, tolling, parking	all modules, SVM	locate and read plates (this project)
Document scanning and text capture	histogram equalisation, adaptive thresholding, rotation for de-skew, resizing	prepare scans for character recognition
Industrial inspection	Canny edges, morphology, connected components, shape filters	find and measure parts and defects
Medical image analysis	histogram equalisation, median filtering, region growing, registration	improve contrast, remove noise, outline regions, align scans of the same patient
Remote sensing and photogrammetry	radial distortion correction, registration, interpolation	align and resample images of the same area
Photo enhancement	histogram matching, unsharp masking, frequency filters, bicubic resizing	improve and enlarge photographs

10 Challenges and Lessons Learned

10.1 Technical challenges

- **No toolboxes.** Re-implementing the standard functions was the main workload. The risk is silent errors, so each function got an independent check (Chapter 8). The slowest primitive, connected-component labelling, was made practical by working on pixel runs with a union–find structure instead of on pixels.
- **Plates with gaps.** A plate such as “ABC 1234” has a wide gap that a narrow morphological closing does not bridge, so the detector boxed only one half. Measuring the detection rate for several closing widths (Table 16) showed a clear optimum and replaced guessing.
- **An idea that did not help.** Adding a local adaptive threshold to the character segmentation looked promising for uneven lighting but lowered the exact-plate accuracy by about six points on the test scenes (Table 17). It stayed as an option and was switched off by default; the measurement was more convincing than the expectation.
- **Honest evaluation.** Because the data are synthetic it would be easy to train and test on the same fonts and report very high accuracy. Holding out four fonts and keeping the test scenes separate from all development decisions about the classifier gives a more sober picture and revealed the strong font dependence.
- **Evidence for the kernel.** The expectation that the RBF kernel is clearly better than a linear one was not borne out: the difference was within the sampling error. We kept the RBF kernel as the default but report the result as it is.
- **Numerical details.** Floating-point effects ($\cos 90° \neq 0$), thresholds relative to the global maximum, and implicit type conversions each caused a bug (Table 28). Tests with exact expected values found them quickly.

10.2 Team collaboration

Describe the challenges of working as a team and how they were solved: dividing the modules, agreeing on the function interfaces (image format, parameter order), exchanging and merging code, finding time to meet, and dealing with disagreements. Add anything specific to your group.

10.3 Lessons learned

- Agree on a common data format first (here: double images in [0, 1]) so that modules written by different people fit together.
- Write the check before trusting a function, and keep the checks running; several bugs appeared only when a function was used in a new way.
- Measure design decisions instead of arguing about them; two of the decisions in the pipeline (closing width, adaptive threshold) were settled by a small experiment.
- State clearly what was and what was not tested. The limitation that no real photographs were used is more useful to a reader than a higher number.

11 Conclusion and Future Work

11.1 Conclusion

The project delivers the requested toolbox: three modules with the techniques listed in the assignment, a graphical interface that offers 48 operations, a licence plate recognition system built from them with a support vector machine written from scratch, and an automatic test suite of 39 checks. All code uses base MATLAB only and is documented in the code (574 comment lines in 61 files).

Measured on synthetic scenes with fonts that the classifier had not seen, the system finds the plate in 99 % of medium-difficulty images and reads 75 % of the plates exactly (83 % when the letter/digit layout is known). The analysis of the errors shows that they come from a few identifiable sources: look-alike characters, font-dependent confusions, and characters that merge with the plate frame after strong rotation. The main open question is performance on real photographs, which this project could not test.

Table 30. Deliverables of the assignment and their status.

Requirement of the assignment	Status
Module 1: histogram processing, spatial and frequency filtering	implemented and tested (Chapter 3)
Module 2: edge detection, segmentation, morphology	implemented and tested (Chapter 4)
Module 3: geometric transformations, interpolation, optional registration	implemented and tested, including the optional registration (Chapter 5)
MATLAB GUI integrating all modules	implemented (Chapter 7)
Project topic: licence plate recognition using SVM	implemented and evaluated on synthetic data (Chapter 6)
Project report	this document
Group presentation (5–10 minutes)	to be prepared by the group

11.2 Future work

- **Test on real photographs.** Collect or obtain a labelled set of plate photographs from the target country, measure the accuracy, retrain with real character crops and tune the detector. This is the most important next step.
- **Perspective correction.** Estimate a homography from the four corners of the plate and warp the plate to a frontal view with the existing reverse-mapping engine, to handle plates photographed at an angle.
- **More robust segmentation.** Separate characters that touch each other or the frame (vertical projection profiles, watershed), and recover lost characters from the expected layout.
- **Confidence-based rejection and layout models.** Use the vote confidence to reject doubtful plates and use a language model of valid plate formats to resolve look-alike characters.
- **Other alphabets** by rendering and training on the characters of the local plate script.
- **Speed.** Compile the detector or restrict it to regions of interest to approach video rate, and compare against a small convolutional network as classifier.

References

Brown, D. C. (1971). Close-range camera calibration. *Photogrammetric Engineering*, 37(8), 855–866.

Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), 679–698.

Cortes, C., and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

Dalal, N., and Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1, 886–893.

Fan, R.-E., Chen, P.-H., and Lin, C.-J. (2005). Working set selection using second order information for training support vector machines. *Journal of Machine Learning Research*, 6, 1889–1918.

Fischler, M. A., and Bolles, R. C. (1981). Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381–395.

Gonzalez, R. C., and Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

Harris, C., and Stephens, M. (1988). A combined corner and edge detector. *Proceedings of the Alvey Vision Conference*, 147–151.

Hsu, C.-W., and Lin, C.-J. (2002). A comparison of methods for multiclass support vector machines. *IEEE Transactions on Neural Networks*, 13(2), 415–425.

Keys, R. G. (1981). Cubic convolution interpolation for digital image processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 29(6), 1153–1160.

Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62–66.

Platt, J. C. (1998). Sequential minimal optimization: a fast algorithm for training support vector machines. Technical Report MSR-TR-98-14, Microsoft Research.

Project assignment: *Smart Image Processing Toolbox for Real-World Applications* (Project.docx), including the topic list “Top 29 Image Processing Projects in 2026 For All Levels + Source Code”.

Appendix A: Reproducing the Results

All numbers and figures of this report can be regenerated. Open MATLAB in the project folder and run:

```
setup_toolbox                  % add the toolbox folders to the path
train_lpr_model                % train the SVMs (about 1 minute), creates models/*.mat
run_tests                      % automatic checks
demo_modules                   % figures of the three modules for one image
demo_lpr                       % recognise a synthetic car photo
LPR_Toolbox                    % start the graphical interface

cd report
make_report_assets             % regenerate every report figure and measurement (about 10
minutes)
```

The report document itself is produced from the measured data by `report/build_report.js` (Node.js with the `docx` package). Random number generators are seeded in every experiment:

Table 31. Random seeds.

Experiment	Seed(s)
Scene for the module figures (Chapters 3–5)	4
Noise images of Module 1	11 (Gaussian), 12 (salt and pepper)
Training set / unseen-font test set / seen-font test set	7 / 207 / 107
End-to-end evaluation at three difficulty levels	123
Error analysis (150 scenes)	321
Detector closing-width experiment	11

Appendix B: List of Automatic Checks

Table 32. The 39 checks of run_tests.m.

#	Check
1	toDouble: uint8 -> [0,1]
2	toDouble: double with small overshoot is left alone
3	padImage replicate
4	histEqualize increases contrast
5	histMatch moves mean towards reference
6	grayTransform negative
7	median beats mean on salt-and-pepper noise
8	Gaussian filter preserves constant image
9	Laplacian sharpening increases edge energy
10	FFT low-pass smooths, high-pass sharpens
11	FFT filters work on RGB
12	labelComponents matches flood-fill reference
13	Sobel/Prewitt/Canny find the step edge
14	Otsu separates a bimodal histogram
15	dilation equals convolution
16	erosion is the dual of dilation
17	opening is anti-extensive, closing extensive
18	fillHoles fills an enclosed hole
19	region growing fills exactly the flat square
20	colour segmentation finds pure yellow
21	rotate 90 deg equals rot90
22	translation moves pixels
23	forward bilinear mapping with identity keeps the image
24	forward mapping leaves holes, reverse mapping does not
25	bilinear interpolates a ramp exactly
26	bicubic interpolates a ramp exactly
27	imResize round trip keeps the image
28	radial distortion round trip
29	strong barrel distortion: finite result, centre recovered, unreachable corners left black
30	feature registration reduces the error
31	phase correlation recovers the shift
32	binary RBF-SVM separates concentric circles
33	SMO keeps sum(alpha*y) = 0
34	HOG descriptor length is 324
35	makeScene returns a plate box inside the image
36	detectPlate finds the plate (IoU > 0.5)
37	pipeline: plate detection >= 90 % on unseen fonts
38	pipeline: character accuracy (format known) >= 90 %
39	pipeline: exact plates (format known) >= 70 %

Appendix C: Operations of the Graphical Interface

Table 33. The 48 operations of the interface and their parameters.

Tab	Operation	Parameters (sliders)
1	Histogram equalisation	–
1	Histogram matching (to reference)	–
1	Brightness / contrast	Contrast gain; Brightness
1	Gamma correction	Gamma
1	Log transform	a
1	Contrast stretching	Low percentile; High percentile
1	Mean filter	Window size
1	Gaussian filter	Window size; Sigma
1	Median filter	Window size
1	Laplacian sharpening	Amount
1	High-pass sharpening	Amount
1	Unsharp masking	Amount
1	Frequency low-pass (Butterworth)	Cut-off
1	Frequency high-pass (Butterworth)	Cut-off
1	Frequency band-pass (Gaussian)	Low cut-off; High cut-off
1	Add Gaussian noise	Sigma
1	Add salt-and-pepper noise	Density
2	Sobel edges	Threshold (0 = Otsu)
2	Prewitt edges	Threshold (0 = Otsu)
2	Canny edges	High threshold (0 = auto); Sigma
2	Otsu threshold	–
2	Adaptive threshold	Window; Offset
2	Region growing (from centre)	Tolerance
2	Colour segmentation: white (HSV)	–
2	Colour segmentation: yellow (HSV)	–
2	Colour segmentation: blue (HSV)	–
2	Colour segmentation: red (RGB range)	–
2	Morphology: dilation	Disk radius
2	Morphology: erosion	Disk radius
2	Morphology: opening	Disk radius
2	Morphology: closing	Disk radius
2	Morphology: gradient (outline)	Disk radius
2	Shape segmentation (plate-like blobs)	–
3	Translation	dx (px); dy (px)
3	Scaling / zoom	Scale
3	Rotation	Angle (deg)
3	Rotation (full canvas)	Angle (deg)
3	Shear	Shear x; Shear y
3	Affine (rotate + scale + shear)	Angle (deg); Scale; Shear
3	Forward mapping (scale, shows holes)	Scale
3	Reverse mapping (scale)	Scale
3	Resize: nearest neighbour	Scale
3	Resize: bilinear	Scale
3	Resize: bicubic	Scale
3	Radial distortion: simulate	k1; k2
3	Radial distortion: correct	k1; k2
3	Image registration (features)	–
3	Image registration (phase corr.)	–